



# RedPyme Gestión

Sistemas Microinformáticos y  
Redes

Marcos Garcia, Iker Carrizal y  
Marcos Padilla

Sara Pulgar Pantoja

13/5/26

IES EL CAÑAVERAL

Departamento de Informática

# Memoria del proyecto

## Documento técnico

### Índice

|                                                                                                  |   |
|--------------------------------------------------------------------------------------------------|---|
| 1.- Introducción.....                                                                            | 1 |
| 1.1.- Descripción y contexto del proyecto .....                                                  | 2 |
| 1.2.- Motivación del proyecto .....                                                              | 2 |
| 1.3.- Beneficios esperados .....                                                                 | 2 |
| 2.- Objetivo/s generales del proyecto.....                                                       | 2 |
| 3.- Contexto actual.....                                                                         | 2 |
| 4.- Análisis de la necesidad .....                                                               | 2 |
| 5.- Diseño de la solución .....                                                                  | 2 |
| 5.1.- Lógico.....                                                                                | 3 |
| 5.2.- Físico .....                                                                               | 3 |
| 6.- Desarrollo de la solución.....                                                               | 3 |
| 6.1.- Tecnologías y herramientas utilizadas (lenguajes de programación, frameworks, etc.). ..... | 3 |
| 7.- Planificación del proyecto.....                                                              | 3 |
| 7.1.- Acciones.....                                                                              | 3 |
| 7.2.- Temporalización y secuenciación.....                                                       | 4 |
| 8.- Pruebas y validación .....                                                                   | 4 |
| 9.- Relación del proyecto con los módulos del ciclo .....                                        | 4 |
| 10.- Conclusiones .....                                                                          | 5 |
| 11.- Proyectos futuros.....                                                                      | 6 |
| 12.- Bibliografía/Webgrafía .....                                                                | 6 |
| 13.- Anexos .....                                                                                | 6 |

## 1.- Introducción

---

### 1.1.- Descripción y contexto del proyecto

NetManager es un panel de control basado en web diseñado para simplificar la administración de servidores Ubuntu en entornos PYME. Se centra en la gestión de servicios críticos de red como DHCP y FTP sin necesidad de usar la terminal.

### 1.2.- Motivación del proyecto

Resolver la dificultad que tienen las pequeñas empresas sin personal técnico para configurar servicios de red complejos, ofreciendo una solución intuitiva y viable.

### 1.3.- Beneficios esperados

Automatización de tareas, reducción de errores de configuración manual y despliegue rápido mediante una imagen ISO personalizada.

## 2.- Objetivo/s generales del proyecto

---

Desarrollar una herramienta integral de administración de sistemas Linux que permita gestionar servicios de red de forma visual y segura.

## 3.- Contexto actual

---

### Estado del arte.-

Actualmente existen paneles como Webmin o Cockpit, pero NetManager busca una experiencia más ligera, personalizada y enfocada en la facilidad de instalación mediante una imagen de sistema propia.

### Conceptos clave.-

Flask (backend), Subprocess (ejecución de comandos), DHCP (protocolo de red), vsftpd (servidor FTP)

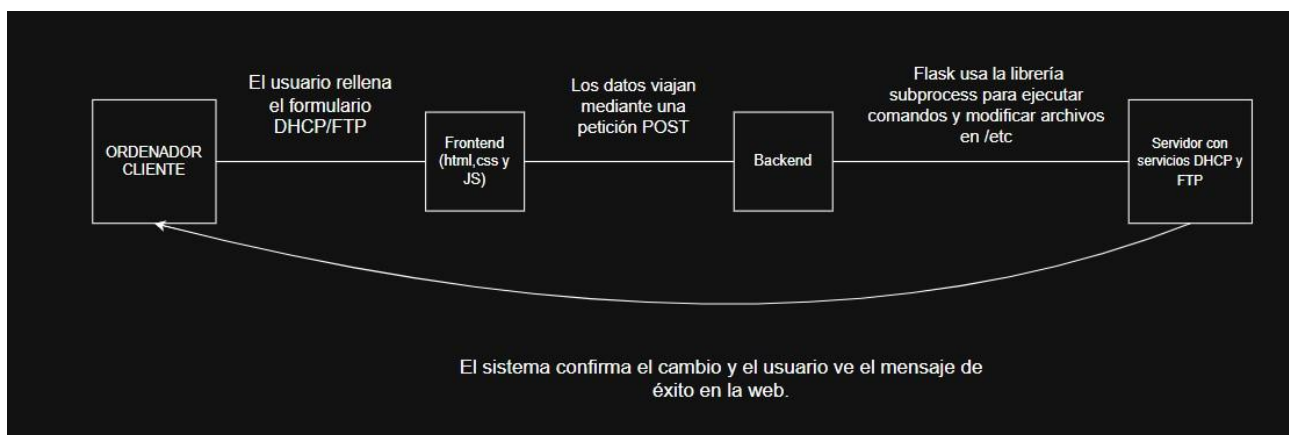
## 4.- Análisis de la necesidad

El proyecto cubre la necesidad de gestionar de forma dinámica y visual servicios que habitualmente requieren edición manual de archivos de texto en /etc , lo cual es propenso a fallos humanos.

## 5.- Diseño de la solución

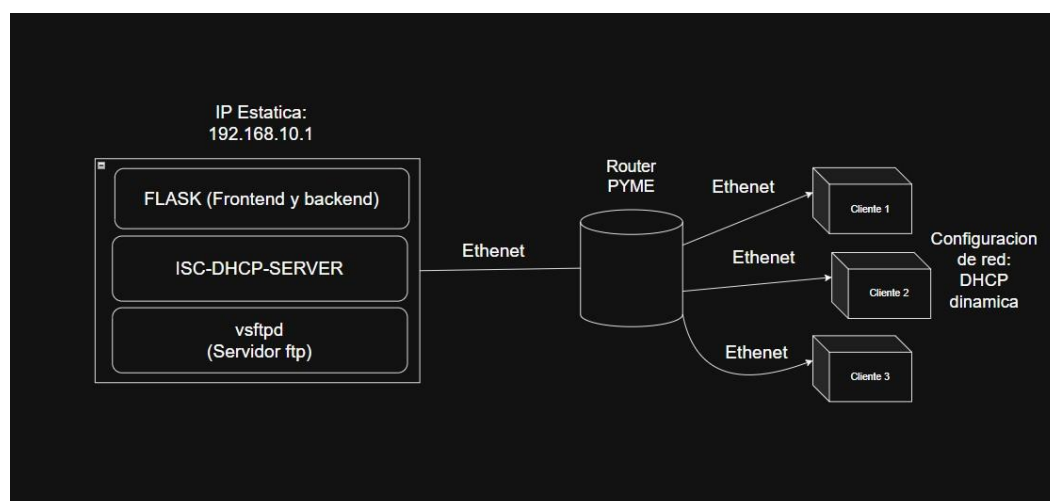
Este apartado tendrá las decisiones que se han tomado para la realización del proyecto:

### 5.1.- Lógico.



### 5.2.- Físico.

Esquema de la arquitectura del sistema.



## 6.- Desarrollo de la solución

Pasos de implementación detallados.

### 6.1.- Tecnologías y herramientas utilizadas (lenguajes de programación, frameworks, etc.).

- **Python/Flask:** Elegido por su ligereza y facilidad para interactuar con scripts de sistema.
- **Visual Studio Code:** Utilizado como entorno de desarrollo principal por su modularidad y extensiones de acceso remoto SSH.
- **VirtualBox:** Infraestructura para el testeo de la ISO y las máquinas cliente.
- **Vsftpd:** Servicio base para FTP por su robustez y seguridad

### 6.2 INFRAESTRUCTURA

Objetivo: Crear el entorno base donde funcionará NetManager PYME.

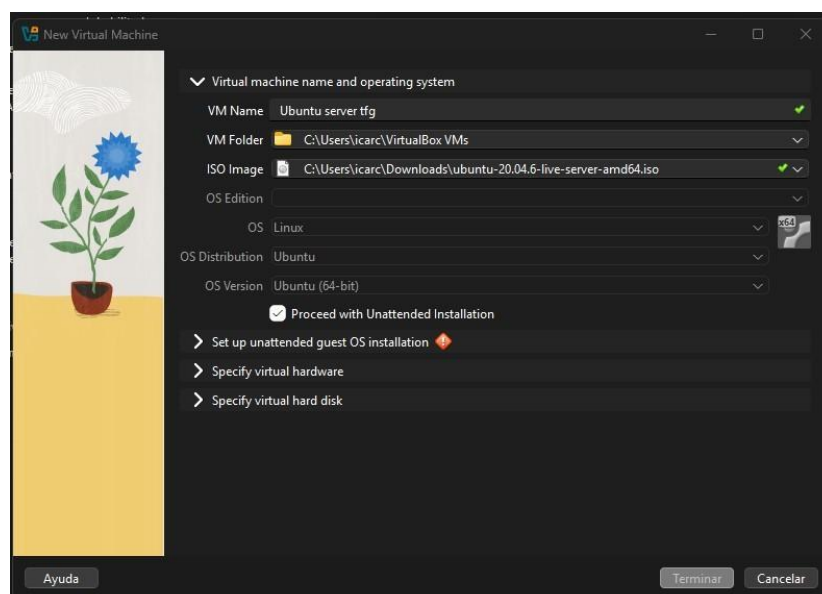
#### 6.2.1 Creación de la máquina Ubuntu Server.

Crear la máquina virtual en VirtualBox

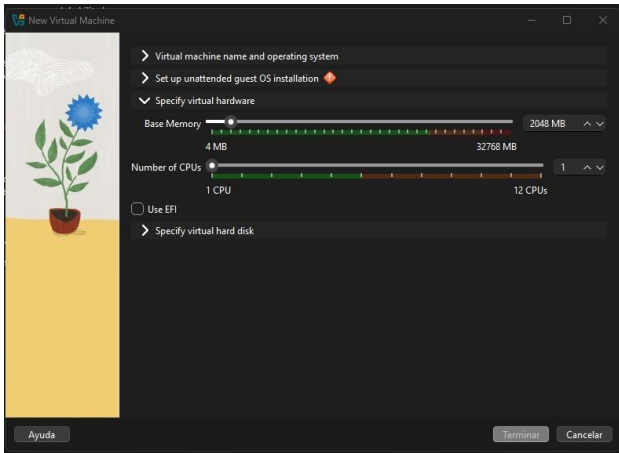
Nombre: Ubuntu Server tfg

Tipo de sistema: Linux

Versión: Ubuntu (64-bit) ISO utilizada: ubuntu-20.04.6-live-server-amd64.iso

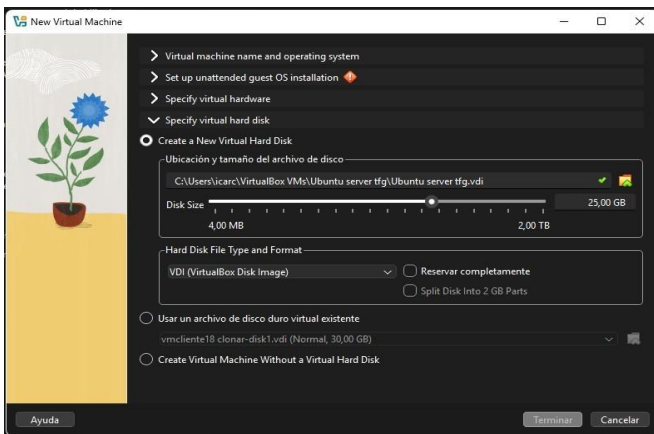


Memoria RAM: entre 2048 MB y 4096 MB, suficiente para los servicios de DNS y DHCP. CPU: 2 núcleos, para mejorar la capacidad de respuesta del sistema.



Disco duro virtual:

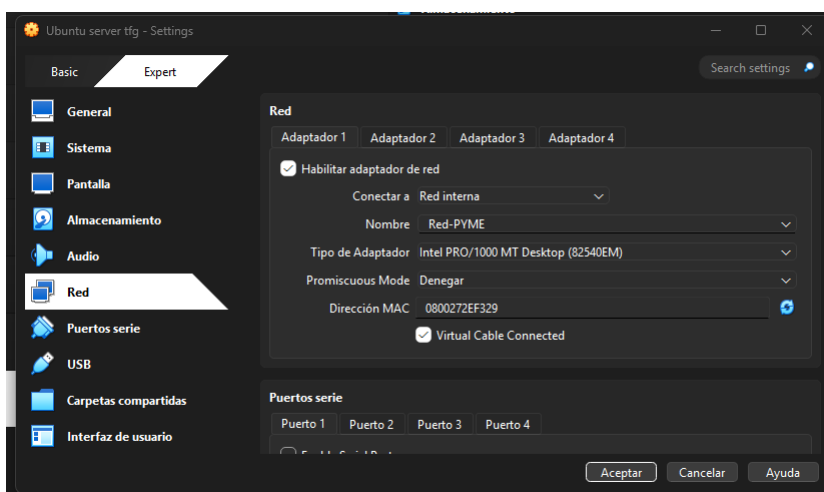
Tamaño: 20 – 40 GB. (en el caso práctico, 25 GB).



Formato: VDI (VirtualBox Disk Image). Tipo de asignación: dinámica, para optimizar el uso de espacio en el host.

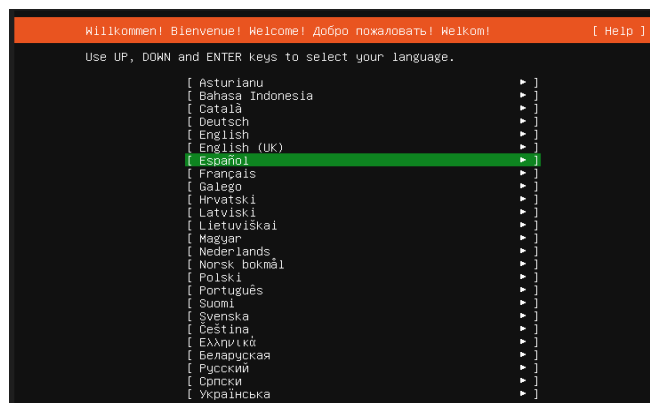
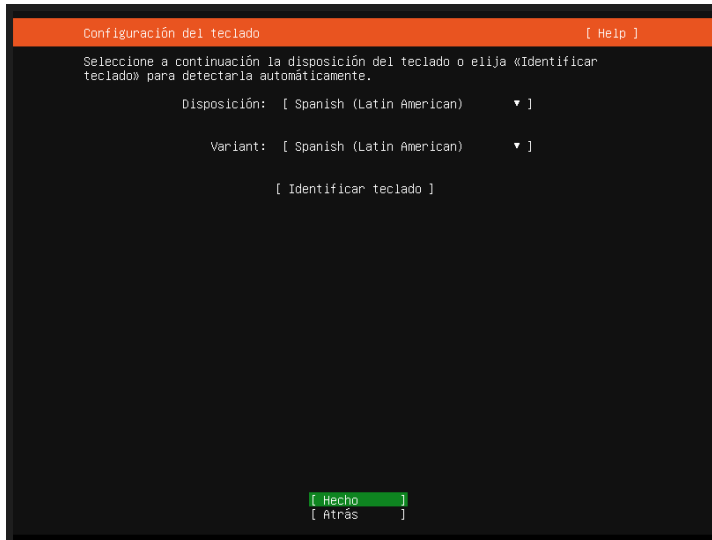
Red.

Adaptador 1 > Red interna. Nombre de la red interna: Red-PYME

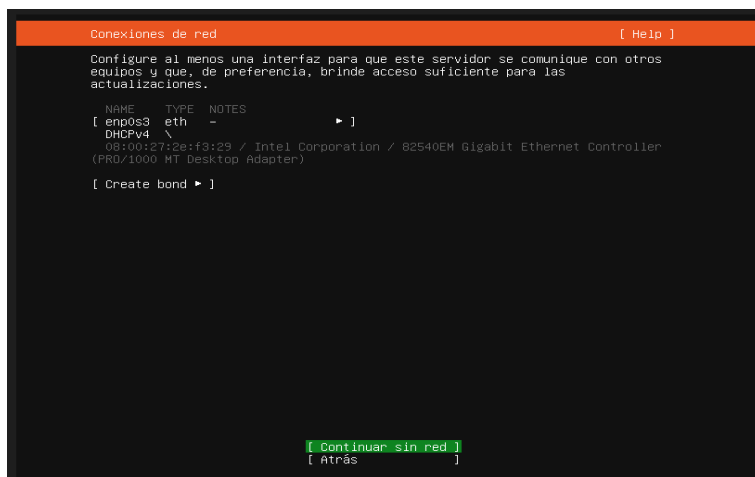


## 6.2.2 Instalación de Ubuntu Server

### 1. Idioma: español + Configuración del teclado



### 2. Omitimos la configuración de red



### 3. Creamos el usuario administrador , le asignamos un nombre al servidor y le asignamos contraseña al usuario.



Configuración de perfil [ Help ]

Proporcione el nombre de usuario y la contraseña que utilizará para acceder al sistema. Puede configurar el acceso SSH en la pantalla siguiente, pero aun se necesita una contraseña para sudo.

Su nombre: tfg

El nombre del servidor: tfg  
El nombre que utiliza al comunicarse con otros equipos.

Elija un nombre de usuario: tfg

Elija una contraseña: \*\*\*\*\*

Confirme la contraseña: \*\*\*\*\*

[ Hecho ]

### 4. Entramos

```
Ubuntu 20.04.6 LTS tfg tty1
tfg login: tfg
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-144-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sun Feb 15 15:36:08 UTC 2026

System load: 0.02      Memory usage: 10%   Processes:   100
Usage of /:  37.1% of 11.21GB Swap usage:   0%      Users logged in: 0

Expanded Security Maintenance for Infrastructure is not enabled.
0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

tfg@tfg:~$
tfg@tfg:~$
```

## 6.2.3 Configuración de red estática en Ubuntu Server

### 1. Revisamos primero las interfaces de red del servidor con ip a:

```
tfg@tfg:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:2e:f3:29 brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:88:04:c2 brd ff:ff:ff:ff:ff:ff
tfg@tfg:~$
```



## 2. Configuramos el archivo de configuración de red con sudo nano /etc/netplan/01-netcfg.yaml de manera temporal

```
GNU nano 4.8 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      addresses:
        - 192.168.10.10/24
      gateway4: 192.168.10.1
      nameservers:
        addresses: [192.168.10.10, 8.8.8.8]
```

## 2. Aplicamos la configuración de red con sudo netplan apply

```
Ubuntu server tfg [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos

tfg@tfg:~$ sudo netplan apply
tfg@tfg:~$ _
```

## 3. Hacemos ip a y comprobamos el resultado para enp0s3:

```
Ubuntu server tfg [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

tfg@tfg:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:2e:f3:29 brd ff:ff:ff:ff:ff:ff
   inet 192.168.10.10/24 brd 192.168.10.255 scope global enp0s3
       valid_lft forever preferred_lft forever
   inet6 fe80::a00:27ff:fe2e:f329/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether 08:00:27:88:04:c2 brd ff:ff:ff:ff:ff:ff
tfg@tfg:~$
```

## 6.2.4 Configurar enp0s8 para obtener IP por DHCP

1. Volvemos a configurar el archivo de configuración de red para instalar paquetes



```
GNU nano 4.8 /etc/netplan/01-netcfg.yaml Modified
network:
version: 2
renderer: networkd
ethernets:
  enp0s3:
    addresses:
      - 192.168.10.1/24
  enp0s8:
    dhcp4: true
```

2. Aplicamos cambios con sudo netplan apply

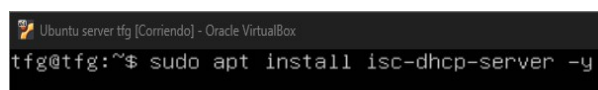
```
tfg@tfg:~$ sudo netplan apply
tfg@tfg:~$
```

3. Hacemos un ping a 8.8.8.8 para comprobar que tenemos conexión a internet

```
tfg@tfg:~$
tfg@tfg:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=12.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=11.2 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=9.69 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=10.4 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=255 time=10.6 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=255 time=12.9 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=255 time=10.8 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=255 time=13.3 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=255 time=13.9 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=255 time=11.6 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=255 time=12.9 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=255 time=11.2 ms
```

## 6.2.5 Instalación y configuración del servicio DHCP

1. Instalamos el servicio DHCP con sudo apt install isc-dhcp-server -y



```
Ubuntu server tfg [Corriendo] - Oracle VirtualBox
tfg@tfg:~$ sudo apt install isc-dhcp-server -y
```

```

Se instalarán los siguientes paquetes NUEVOS:
  isc-dhcp-server libirs-export161 libiscfg-export163
0 actualizados, 3 nuevos se instalarán, 0 para eliminar y 207 no actualizados.
Se necesita descargar 520 kB de archivos.
Se utilizarán 1.866 kB de espacio de disco adicional después de esta operación.
Des:1 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libiscfg-export163 amd64 1:9.11.16+dfsg-3~ubuntu1 [45,9 kB]
Des:2 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libirs-export161 amd64 1:9.11.16+dfsg-3~ubuntu1 [16,6 kB]
Des:3 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 isc-dhcp-server amd64 4.4.1-2.1ubuntu5.20.04.5 [455 kB]
Descargados 520 kB en 1s (582 kB/s)
Preconfigurando paquetes ...
Seleccionando el paquete libiscfg-export163 previamente no seleccionado.
(Leyendo la base de datos ... 69582 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../libiscfg-export163_1x3a9.11.16+dfsg-3~ubuntu1_amd64.deb ...
Desempaquetando libiscfg-export163 (1:9.11.16+dfsg-3~ubuntu1) ...
Seleccionando el paquete libirs-export161 previamente no seleccionado.
Preparando para desempaquetar .../libirs-export161_1x3a9.11.16+dfsg-3~ubuntu1_amd64.deb ...
Desempaquetando libirs-export161 (1:9.11.16+dfsg-3~ubuntu1) ...
Seleccionando el paquete isc-dhcp-server previamente no seleccionado.
Preparando para desempaquetar .../isc-dhcp-server_4.4.1-2.1ubuntu5.20.04.5_amd64.deb ...
Desempaquetando isc-dhcp-server (4.4.1-2.1ubuntu5.20.04.5) ...
Configurando libiscfg-export163 (1:9.11.16+dfsg-3~ubuntu1) ...
Configurando libirs-export161 (1:9.11.16+dfsg-3~ubuntu1) ...
Configurando isc-dhcp-server (4.4.1-2.1ubuntu5.20.04.5) ...
Generating /etc/default/isc-dhcp-server...
Created symlink /etc/systemd/system/multi-user.target.wants/isc-dhcp-server.service → /lib/systemd/system/isc-dhcp-server.service.
Created symlink /etc/systemd/system/multi-user.target.wants/isc-dhcp-server6.service → /lib/systemd/system/isc-dhcp-server6.service.
Procesando disparadores para man-db (2.9.1-1) ...
Procesando disparadores para libc-bin (2.31-0ubuntu9.9) ...
Procesando disparadores para systemd (245.4-4ubuntu3.20) ...
tf@tf:~$

```

2. Editamos el archivo de iniciación del servicio DHCP con `sudo nano /etc/default/isc-dhcp-server`

```

Ubuntu server tfg [Corriendo] - Oracle VirtualBox
tf@tf:~$ sudo nano /etc/default/isc-dhcp-server

```

```

GNU nano 4.8 /etc/default/isc-dhcp-server
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4_CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4_PID=/var/run/dhcpd.pid
#DHCPDv6_PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="enp0s3"
INTERFACESv6=""

```

Lo editamos de tal forma que el servicio DHCP escuche la primera interfaz (la interna)

```

Ubuntu server tfg [Corriendo] - Oracle VirtualBox
tf@tf:~$ sudo nano /etc/dhcp/dhcpd.conf

```

3. Editamos el archivo de configuración principal DHCP con `sudo nano /etc/dhcp/dhcpd.conf`. Lo editamos de tal forma que en la subred 192.168.10.0 otorgue IPs a los equipos dentro de la red privada, con un rango desde 192.168.10.50 hasta 192.168.10.60

```
GNU nano 4.8 /etc/dhcp/dhcpd.conf
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.50 192.168.10.60;
    option routers 192.168.10.1;
    option domain-name-servers 8.8.8.8, 8.8.4.4;
}
```

#### 4. Reiniciar y comprobar DHCP

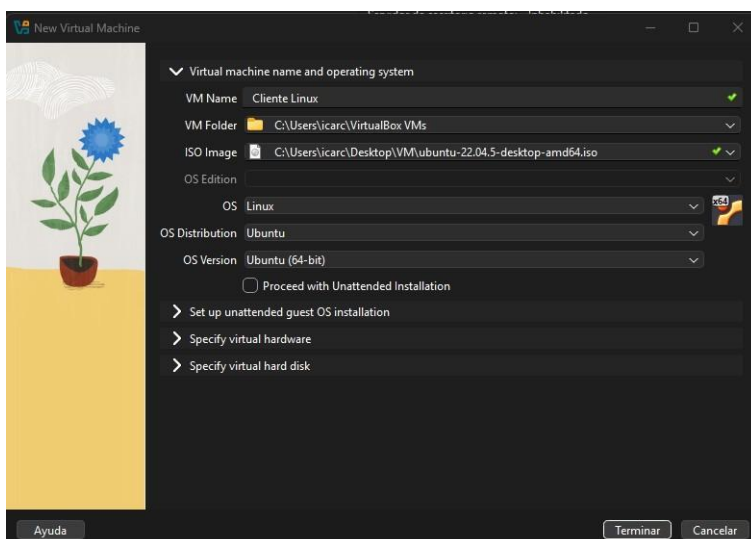
Reiniciamos el servicio DHCP con `sudo systemctl restart isc-dhcp-server` y comprobamos su estado con `sudo systemctl status isc-dhcp-server`

```
tfg@tfg:~$ sudo systemctl restart isc-dhcp-server
tfg@tfg:~$ sudo systemctl status isc-dhcp-server
* isc-dhcp-server.service - ISC DHCP IPv4 server
   Loaded: loaded (/lib/systemd/system/isc-dhcp-server.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-02-18 16:46:24 UTC; 2s ago
     Docs: man:dhcpd(8)
    Main PID: 93076 (dhcpd)
      Tasks: 4 (limit: 2271)
     Memory: 4.4M
    CGroup: /system.slice/isc-dhcp-server.service
            └─93076 dhcpd -user dhcpd -group dhcpd -f -4 -pf /run/dhcp-server/dhcpd.pid -cf /etc/dh
```

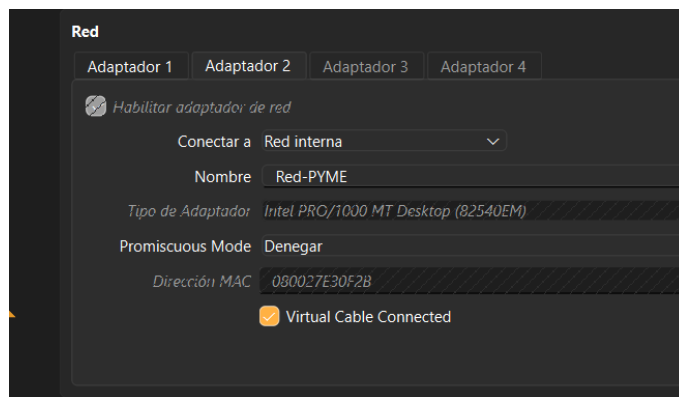
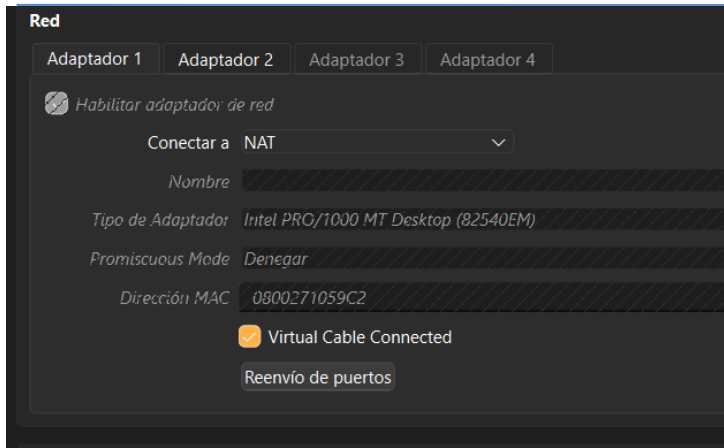
```
feb 18 16:46:24 tfg dhcpd[93076]: PID file: /run/dhcp-server/dhcpd.pid
feb 18 16:46:24 tfg dhcpd[93076]: Wrote 0 leases to leases file.
feb 18 16:46:24 tfg sh[93076]: Wrote 0 leases to leases file.
feb 18 16:46:24 tfg dhcpd[93076]: Listening on LPF/enp0s3/08:00:27:2e:f3:29/192.168.10.0/24
feb 18 16:46:24 tfg sh[93076]: Listening on LPF/enp0s3/08:00:27:2e:f3:29/192.168.10.0/24
feb 18 16:46:24 tfg sh[93076]: Sending on LPF/enp0s3/08:00:27:2e:f3:29/192.168.10.0/24
feb 18 16:46:24 tfg sh[93076]: Sending on Socket/fallback/fallback-net
feb 18 16:46:24 tfg dhcpd[93076]: Sending on LPF/enp0s3/08:00:27:2e:f3:29/192.168.10.0/24
feb 18 16:46:24 tfg dhcpd[93076]: Sending on Socket/fallback/fallback-net
feb 18 16:46:24 tfg dhcpd[93076]: Server starting service.
lines 1-20/20 (END)
```

### 6.2.6 Crear un cliente Linux y probar servicios

1. Creamos una máquina cliente en VirtualBox. Con el nombre Cliente Linux , usando la versión Ubuntu-22.04.05 desktop



2. Configuramos los adaptadores de red , en el primer adaptador usamos NAT para que tenga acceso a internet. Y en el segundo la configuramos para que se conecte a la red privada red-PYME , la misma en la que esta en servidor con el servicio DHCP



- Una vez dentro del cliente abrimos el terminal usamos ip a para comprobar que el servicio DHCP nos a asignado una IP correcta dentro del rango establecido (192.168.10.50-192.168.10.60)

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:10:59:c2 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86244sec preferred_lft 86244sec
   inet6 fd17:625c:f037:2:cd11:c6f8:62ea:f528/64 scope global temporary dynamic
       valid_lft 86245sec preferred_lft 14245sec
   inet6 fd17:625c:f037:2:4e68:564a:deee:cba/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86245sec preferred_lft 14245sec
   inet6 fe80::b22a:1105:c48f:28d5/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:e3:0f:2b brd ff:ff:ff:ff:ff:ff
   inet 192.168.10.53/24 brd 192.168.10.255 scope global dynamic noprefixroute enp0s8
       valid_lft 592sec preferred_lft 592sec
   inet6 fe80::1006:c6ab:a9ab:7ec8/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

### 6.2.7 Instalación de paquetes para programación

Instalamos python3 en el servidor

```
Ubuntu server tfg [Corriendo] - Oracle VM VirtualBox  
tfg@tfg:~$ sudo apt install python3-flask -y
```

Instalamos el servicio openSSH

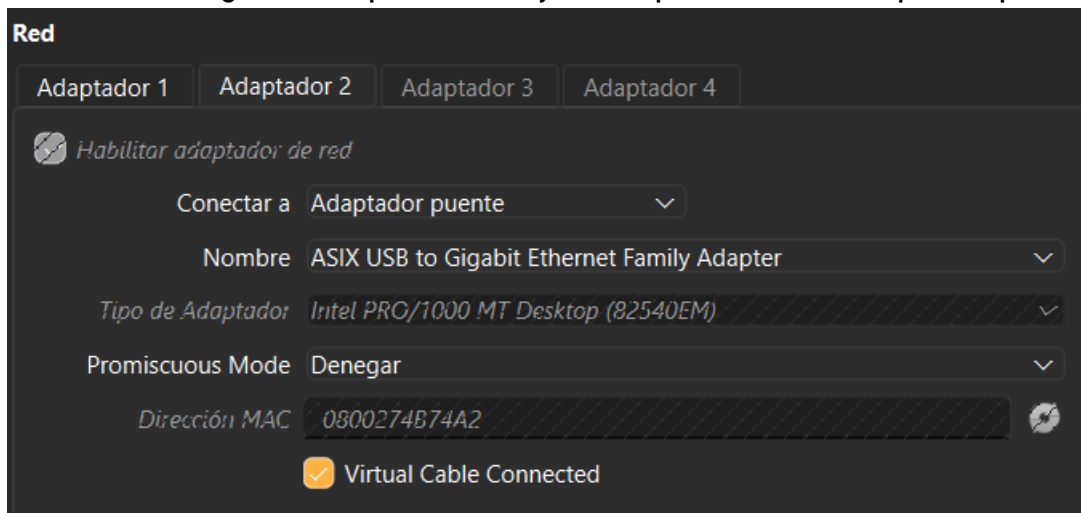
```
Ubuntu server tfg [Corriendo] - Oracle VM VirtualBox  
tfg@tfg:~$ sudo apt install openssh-server -y
```

Instalamos el servicio FTP

```
tfg@tfg:~$ sudo apt install vsftpd -y
```

### 6.2.8 Configuración definitiva de red

Quitamos el segundo adaptador NAT y lo remplazamos un adaptador puente



Configuramos el archivo de configuración de red

```
tfg@tfg:/$ sudo nano /etc/netplan/01-netcfg.yaml
```

Configuramos la segunda interfaz (la puente) para que tenga la ip 192.168.1.35

```
GNU nano 4.8 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      addresses:
        - 192.168.10.1/24
    enp0s8:
      addresses:
        - 192.168.1.35/24
```

Aplicamos la el plan de red

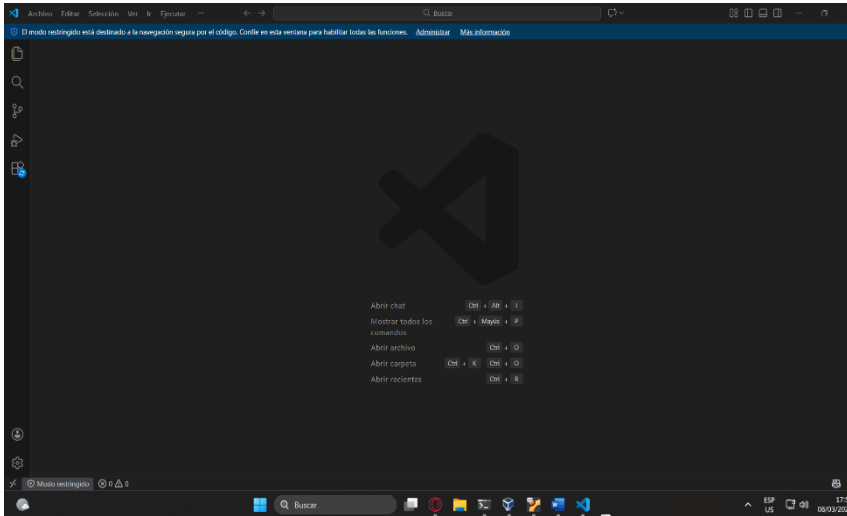
```
Ubuntu server tfg [Corriendo] - Oracle VM VirtualBox
tfg@tfg:~$ sudo netplan apply
tfg@tfg:~$
```

Usamos “ip a” para confirmar que la ip privada de la maquina servidor ha sido asignada mediante dhcp por parte del router. En este caso la IP asignada es 192.168.1.35

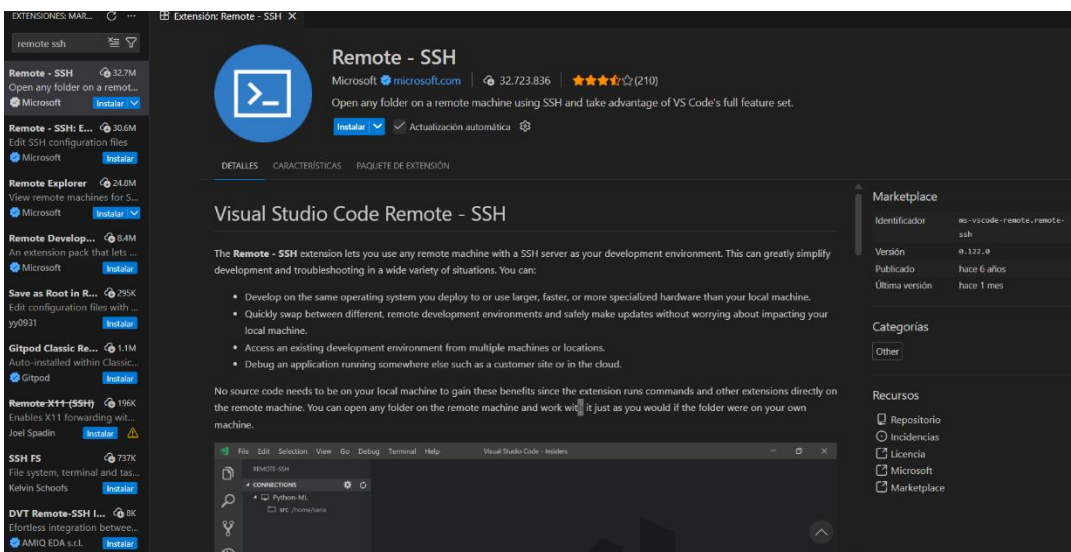
```
tfg@tfg:/etc/netplan$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:61:b3:c1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.1/24 brd 192.168.10.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe61:b3c1/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:4b:74:a2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.35/24 brd 192.168.1.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe4b:74a2/64 scope link
        valid_lft forever preferred_lft forever
tfg@tfg:/etc/netplan$
```

## 6.3 Programación del frontend y backend

Abrimos visual studio code

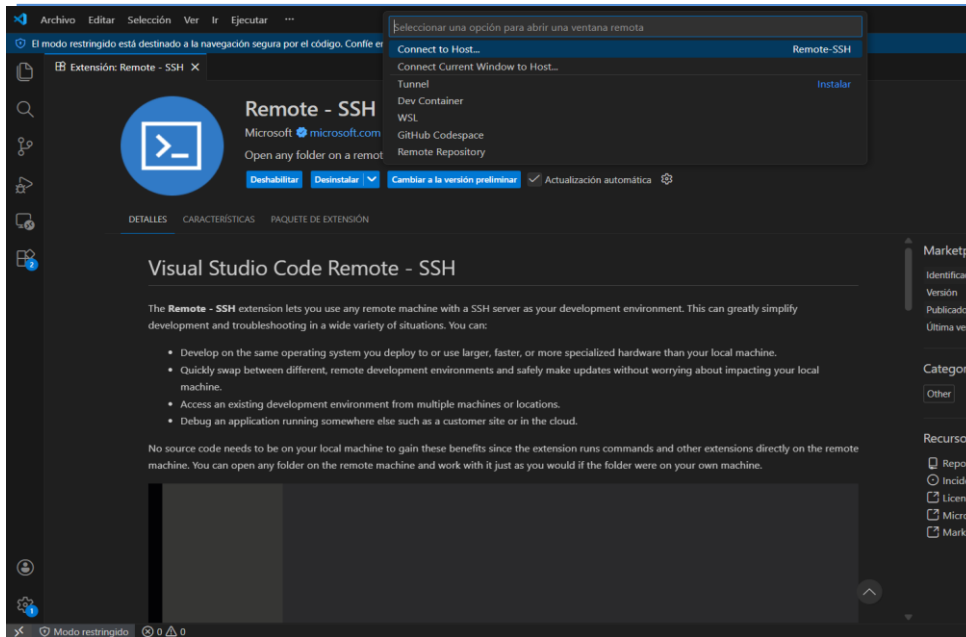


Instalo la extensión remote SSH para poder programar desde visual estudio en la maquina servidor

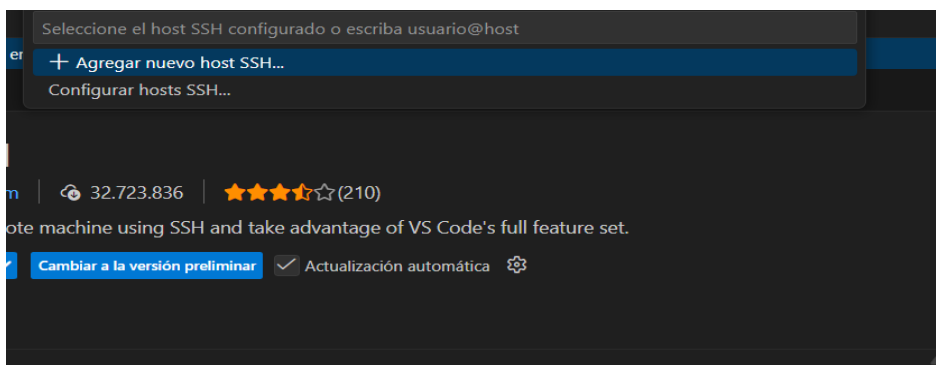


Abajo a la izquierda hacemos clic al símbolo ><, y hacemos clic en Connect to Host...

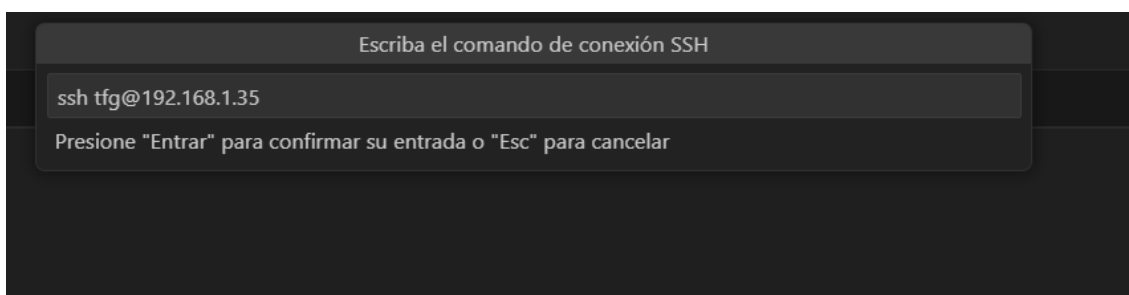




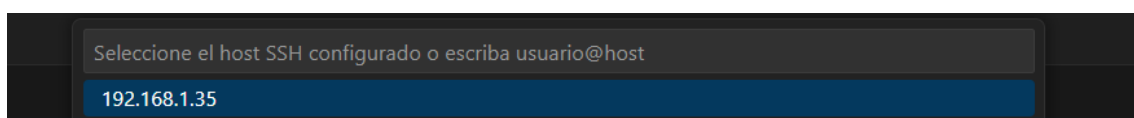
## Agregar nuevo host SSH...



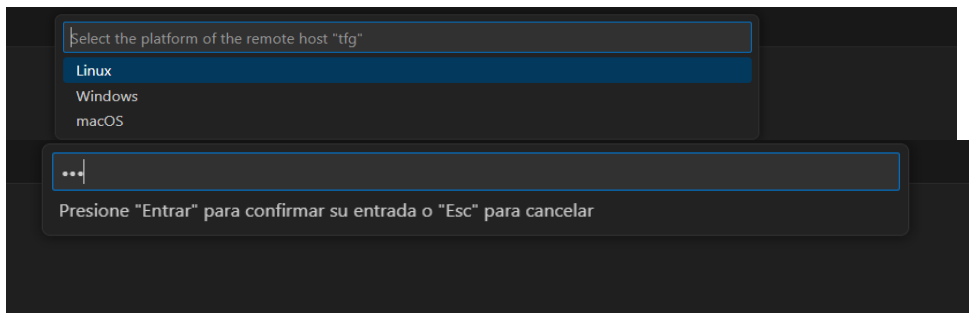
## Configuramos un nuevo host para conectarnos por SSH al usuario tfg a la ip 192.168.1.35



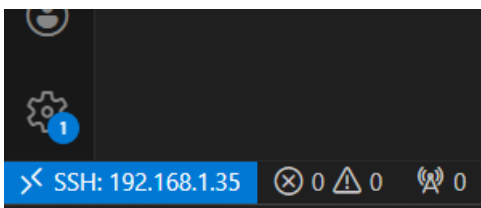
## Seleccionamos el nuevo host configurado



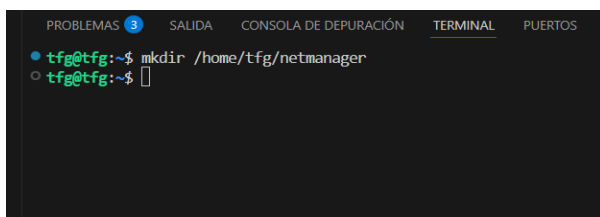
Seleccionamos Linux como la plataforma donde este alojado el usuario tfg



Ya estamos conectados por SSH al servidor



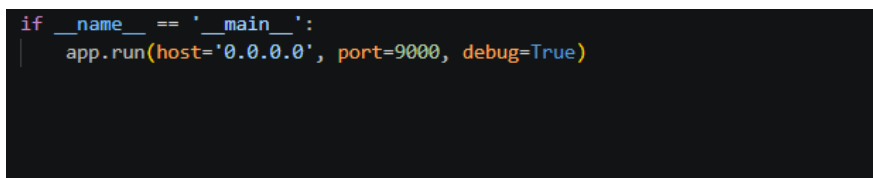
Creamos una carpeta desde el terminal de VisualStudio



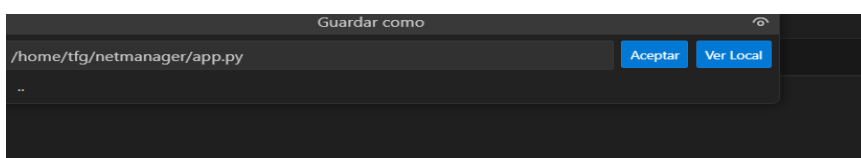
### 6.3.2 Creación del Backend

Inicialización y Apertura de Red (\_main\_)

Arranca el programa , configurar el parámetro `host='0.0.0.0'` es muy importante para el TFG, ya que le ordena a Flask escuchar en todas las interfaces de red físicas y virtuales de la máquina, lo que habilita a los miembros del tribunal o tutores a conectarse al panel introduciendo la IP del servidor desde sus ordenadores externos a través del puerto 9000. `debug=True` habilita el modo de depuración en caliente para desarrolladores.



Le asignamos el nombre de app.py al archivo.



Ejecutamos el archivo app.py para que la web sea visible desde un navegador dentro de la misma red privada

```
tf@tf:~/netmanager$ python3 app.py
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 273-101-643
```

Nos conectamos a la ip 192.168.1.35:9000 y funciona correctamente

### Backend de Marcos García

Si estás lees esto, Python y el servidor Ubuntu se están comunicando perfectamente.

## 6.3.3 Creación de funciones

Añadimos funciones para ver el estado de SSH y recursos del sistema

Aquí hay dos funciones , inicio y estado\_ssh

La función inicio se ejecuta únicamente cuando el usuario esta en '/', y imprime dos textos clicables para ver el estado del servicio ssh y para ver el estado de la ram del sistema

La función estado-ssh se ejecuta únicamente cuando el usuario está en '/estado-ssh'

Esta función ejecuta el comando systemctl status ssh y lo guarda en la variable comando. Después ejecuta el comando , y lo guarda en la variable resultado con formato de terminal en el programa con texto puro en vez de en binario.

En la siguiente línea guarda solo el resultado devuelto por el terminal en la variable salida con el atributo .stdout

En la siguiente línea devuelve la variable salida (lo que nos ha devuelto el terminal con el estado de SSH limpio) y al lado un texto clickeable para volver al inicio.

Y por último agregamos una excepción para que si sucede cualquier error devuelve el error junto a un texto indicando que algo ha fallado

```

1 import subprocess
2 from flask import Flask
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def inicio():
8     return """
9     <h1>Panel de Control NetManager</h1>
10    <ul>
11        <li><a href="/estado-ssh">Ver estado del servicio SSH</a></li>
12        <li><a href="/sistema">Ver recursos del sistema (RAM y Disco)</a></li>
13    </ul>
14    """
15
16 @app.route('/estado-ssh')
17 def estado_ssh():
18     try:
19         comando = ['systemctl', 'status', 'ssh']
20         resultado = subprocess.run(comando, capture_output=True, text=True)
21         salida = resultado.stdout
22
23         return f"<h2>Estado del SSH:</h2><pre>{salida}</pre><br><a href='/'>Volver al inicio</a>"
24
25     except Exception as e:
26         return f"Ups, algo falló: {e}"
27
28 @app.route('/sistema')
29 def sistema():
30     try:
31

```

La función estado-ssh se ejecuta únicamente cuando el usuario está en '/sistema'

Esta función ejecuta el free -h y lo guarda en la variable resultado\_ram. Después ejecuta el comando , y lo guarda en la variable resultado con formato de terminal en el programa con texto puro en vez de en binario.

En la línea siguiente guarda solo el resultado devuelto por el terminal en la variable ram con el atributo .stdout

En la siguiente línea devuelve la variable ram (lo que nos ha devuelto el terminal con el estado de la ram del servidor) y al lado un texto clickeable para volver al inicio.

Y por último agregamos una excepción para que si sucede cualquier error devuelve el error junto a un texto indicando que algo ha fallado.

```

27         return f"Ups, algo falló: {e}"
28
29 @app.route('/sistema')
30 def sistema():
31     try:
32         resultado_ram = subprocess.run(['free', '-h'], capture_output=True, text=True)
33         ram = resultado_ram.stdout
34
35         return f"<h2>Memoria RAM:</h2><pre>{ram}</pre><br><a href='/'>Volver al inicio</a>"
36     except Exception as e:
37         return f"Ups, algo falló: {e}"
38
39 if __name__ == '__main__':
40     app.run(host='0.0.0.0', port=5000, debug=True)

```

Así quedaría el panel principal en '/'

## Panel de Control NetManager

- [Ver estado del servicio SSH](#)
- [Ver recursos del sistema \(RAM y Disco\)](#)

Aquí se puede ver el estado de SSH en '/estado-ssh'

#### Estado del SSH:

```
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-03-08 18:03:18 UTC; 40min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 2468 (sshd)
     Tasks: 1 (limit: 2250)
    Memory: 3.4M
   CGroup: /system.slice/ssh.service
           └─2468 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

mar 08 18:12:29 tfg sshd[3242]: Accepted password for tfg from 192.168.1.2 port 27964 ssh2
mar 08 18:12:29 tfg sshd[3242]: pam_unix(sshd:session): session opened for user tfg by (uid=0)
mar 08 18:17:41 tfg sshd[4018]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 t
mar 08 18:17:43 tfg sshd[4018]: Failed password for tfg from 192.168.1.2 port 43542 ssh2
mar 08 18:17:49 tfg sshd[4018]: Accepted password for tfg from 192.168.1.2 port 43542 ssh2
mar 08 18:17:49 tfg sshd[4018]: pam_unix(sshd:session): session opened for user tfg by (uid=0)
mar 08 18:23:50 tfg sshd[4531]: Accepted password for tfg from 192.168.1.2 port 18892 ssh2
mar 08 18:23:50 tfg sshd[4531]: pam_unix(sshd:session): session opened for user tfg by (uid=0)
mar 08 18:24:04 tfg sshd[4745]: Accepted password for tfg from 192.168.1.2 port 38968 ssh2
mar 08 18:24:04 tfg sshd[4745]: pam_unix(sshd:session): session opened for user tfg by (uid=0)
```

[Volver al inicio](#)

Y por ultimo el uso de la memoria RAM en 'sistema'

#### Memoria RAM:

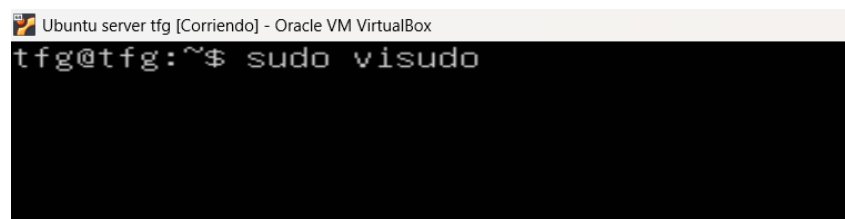
|       | total | used  | free  | shared | buff/cache | available |
|-------|-------|-------|-------|--------|------------|-----------|
| Mem:  | 1,9Gi | 891Mi | 392Mi | 1,0Mi  | 687Mi      | 912Mi     |
| Swap: | 2,0Gi | 5,0Mi | 2,0Gi |        |            |           |

[Volver al inicio](#)

### 6.3.4 Configuración de /etc/sudoers

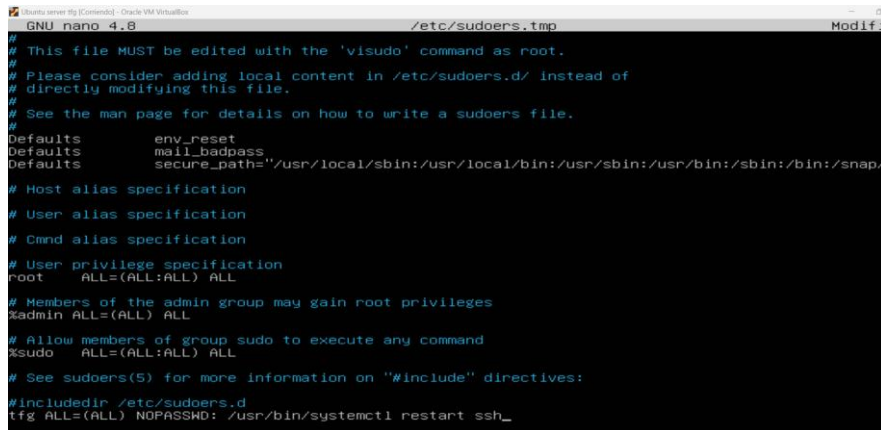
Para que desde el panel de control podamos configurar reiniciar servicios , necesitamos permisos de sudo , pero con Python no podemos hacerlo. Para ello vamos a configurar que se puedan reiniciar el servicio SSH sin permisos de sudo.

Usamos el comando sudo visudo



Configuramos el archivo /etc/sudoers.tmp y le agregamos esta línea para que no necesitemos usar contraseña al ejecutar los comandos y para acceder a archivos que se posicionan a la derecha de la línea de código:

```
tfg ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart ssh
```



```
GNU nano 4.8 /etc/sudoers.tmp
# This file MUST be edited with the 'visudo' command as root.
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
# See the man page for details on how to write a sudoers file.
#
Defaults    env_reset
Defaults    mail_badpass
Defaults    secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/
# Host alias specification
# User alias specification
# Cmnd alias specification
# User privilege specification
root    ALL=(ALL:ALL) ALL
# Members of the admin group may gain root privileges
%admin    ALL=(ALL) ALL
# Allow members of group sudo to execute any command
%sudo    ALL=(ALL:ALL) ALL
# See sudoers(5) for more information on "#include" directives:
#includedir /etc/sudoers.d
tfg    ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart ssh_
```

### 6.3.5 Función de reinicio

La función reiniciar-ssh se ejecuta únicamente cuando el usuario está en '/reiniciar-ssh'

Esta función ejecuta el sudo systemctl restart ssh y lo guarda en la variable comando. Después ejecuta el comando , y lo guarda en la variable resultado con formato de terminal en el programa con texto puro en vez de en binario.

En la siguiente línea devuelve un texto indicando que el servicio ssh se ha reiniciado con éxito con permisos de sudo y al lado un texto clickeable para volver al inicio.

Y por último agregamos una excepción para que si sucede un error al ejecutar el comando devuelve el error exacto que ha ocurrido junto a un texto indicando que algo ha fallado, junto a un botón clickeable para volver al inicio.

```
@app.route('/reiniciar-ssh')
def reiniciar_ssh():
    try:
        comando = ['sudo', 'systemctl', 'restart', 'ssh']

        resultado = subprocess.run(comando, capture_output=True, text=True, check=True)

        return """
        <h2>Servicio SSH reiniciado con éxito</h2>
        <p>El código de Python ha ejecutado el reinicio como administrador.</p>
        <a href="/">Volver al inicio</a>
        """

    except subprocess.CalledProcessError as e:
        return f"<h2>Error al reiniciar:</h2><p>{e.stderr}</p><br><a href="/">Volver al inicio</a>"
```

Así se vería esta nueva función en el menú principal '/'

## Panel de Control NetManager

- [Ver estado del servicio SSH](#)
- [Ver recursos del sistema](#)
- **[REINICIAR SERVICIO SSH](#)**

Y así se vería tras reiniciar el servicio SSH con éxito

### Servicio SSH reiniciado con éxito

El código de Python ha ejecutado el reinicio como administrador.

[Volver al inicio](#)

## 6.3.6 Configuración para reinicios de servicios

Ponemos que ahora se pueda reiniciar los servicios bind9 y isc-dhcp-server sin permisos sudo

```
#includedir /etc/sudoers.d
tfg ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart ssh
tfg ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart bind9
tfg ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart isc-dhcp-server
```

## 6.3.7 Función de reinicio de servicios

Editamos la función reiniciar\_servicio para que también se pueda reiniciar los servicios bind9 (**descartado**) , isc-dhcp-server y SSH.

La función servicio se ejecuta únicamente cuando el usuario está en '/estado/[servicio cliqueado]'

Primero creamos un diccionario , nombre\_reales. Dentro del diccionario asignamos a las keys (los nombres genéricos de servicios) , valores (los nombres técnicos de los servicios)

Después le asignamos a la variable nombre\_ubuntu el valor de nombres\_reales una vez comprobado su valor pasando por el diccionario.

En la siguiente línea ejecuta el systemctl status [nombre servicio] y lo guarda en la variable resultado. Después prueba a ejecutar el comando , y lo guarda en la variable resultado con formato de terminal en el programa con texto puro en vez de en binario.

En la siguiente línea devuelve un texto indicando que el servicio ssh se ha reiniciado con éxito con permisos de sudo y al lado un texto clickeable para volver al inicio.

Y por último agregamos una excepción para que si sucede un error al ejecutar el comando devuelve el error exacto que ha ocurrido junto a un texto indicando que algo ha fallado, junto a un botón clickeable para volver al inicio.

```
@app.route('/estado/<servicio>')
def ver_estado(servicio):
    nombres_reales = {"ssh": "ssh", "dns": "bind9", "dhcp": "isc-dhcp-server"}
    nombre_ubuntu = nombres_reales.get(servicio)

    try:
        resultado = subprocess.run(['systemctl', 'status', nombre_ubuntu], capture_output=True, text=True)
        return f"<h2>Estado de {servicio.upper()}:</h2><pre>{resultado.stdout}</pre><br><a href='/'>Volver al panel</a>"
    except Exception as e:
        return f"Error leyendo {servicio}: {e}"

@app.route('/reiniciar/<servicio>')
def reiniciar_servicio(servicio):
    nombres_reales = {"ssh": "ssh", "dns": "bind9", "dhcp": "isc-dhcp-server"}
    nombre_ubuntu = nombres_reales.get(servicio)

    try:
        comando = ['sudo', 'systemctl', 'restart', nombre_ubuntu]
        subprocess.run(comando, capture_output=True, text=True, check=True)

        return f"<h2>Servicio {servicio.upper()} reiniciado con éxito</h2><br><a href='/'>Volver al panel</a>"
    except subprocess.CalledProcessError as e:
        return f"<h2>Error al reiniciar {servicio.upper()}:</h2><pre>{e.stderr}</pre><br><a href='/'>Volver al panel</a>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Así quedaría el resultado:

## Panel de Control NetManager

### Monitorización:

- [Ver estado SSH](#)
- [Ver estado DNS \(Bind9\)](#)
- [Ver estado DHCP](#)

### Gestión (Peligro):

- [Reiniciar SSH](#)
- [Reiniciar DNS](#)
- [Reiniciar DHCP](#)

```
# --- RUTA DE CONFIGURACION DHCP ---
@app.route('/configurar/dhcp', methods=['GET', 'POST'])
def configurar_dhcp():
    if request.method == 'GET':
        return """
        <h2>Configuracion del Servidor DHCP</h2>
        <form action="/configurar/dhcp" method="POST">
            <label>Subred (Ej: 192.168.10.0):</label><br>
            <input type="text" name="subred" required><br><br>

            <label>Mascara (Ej: 255.255.255.0):</label><br>
            <input type="text" name="mascara" required><br><br>

            <label>Rango Inicio (Ej: 192.168.10.50):</label><br>
            <input type="text" name="rango_inicio" required><br><br>

            <label>Rango Fin (Ej: 192.168.10.100):</label><br>
            <input type="text" name="rango_fin" required><br><br>

            <label>Puerta de Enlace (Router):</label><br>
            <input type="text" name="router" required><br><br>

            <button type="submit">Guardar y Aplicar Configuracion</button>
        </form>
        <br><a href="/">Volver al panel</a>
        """
    else:
        # Configuración DHCP
        subnet {subred} netmask {mascara} {
            range {rango_inicio} {rango_fin};
            option routers {router};
            option domain-name-servers 8.8.8.8, 8.8.4.4;
        }
        """
        try:
            with open('/etc/dhcp/dhcpd.conf', 'w') as archivo:
                archivo.write(nueva_configuracion)

            subprocess.run(['sudo', 'systemctl', 'restart', 'isc-dhcp-server'], check=True)
            return "<h2>DHCP configurado y reiniciado con éxito.</h2><a href='/'>Volver al panel</a>"
        except Exception as e:
            return f"<h2>Error critico:</h2><p>{e}</p><a href='/'>Volver al panel</a>"
```



### 6.3.8 ACTUALIZACIONES GENERALES DEL PROGRAMA Y LA WEB

Dentro del archivo `/etc/sudoers` hemos añadido nuevos comandos para que puedan usar usados sin permisos de sudo. Esto es fundamental , ya que por seguridad el archivo `Py` no tiene que tener derechos de sudo totales.

Hemos añadido las siguientes:

- Useradd: Comando para añadir nuevos usuarios , lo usaremos en el módulo ftp
- Chpasswd: Comando para añadir contraseña a los usuarios creados , lo usaremos en el módulo ftp
- Userdel: Comando para borrar usuarios , lo usaremos para el módulo ftp
- Systemctl: Comando para ver estados , logs y reiniciar servicios
- Mv: Comando con el que moveremos los archivos temporales de configuración creados en `/tmp` a los directorios correspondientes de cada servicio.

```
#includedir /etc/sudoers.d
#setfc ALL=(ALL) NOPASSWD: /usr/sbin/useradd, /usr/sbin/chpasswd, /usr/sbin/userdel, /usr/bin/systemctl

#includedir /etc/sudoers.d
systemctl, /usr/bin/mv
```

Monitorización de servicios ('/' y '/servicios'):

Con `subprocess.run(['systemctl', 'is-active', servicio])`, el programa consulta el estado de los servicios `isc-dhcp-server`, `vsftpd` y `ssh`. El estado se mete en el frontend para enseñar el panel con el estado. Además, añadimos donde un sistema de excepciones para mostrar también cuando un sistema no está activo en el panel, mostrando "inactive" si al consultar el estado, al sistema falla.

```
@app.route('/')
def inicio():
    try:
        status_dhcp = subprocess.run(['systemctl', 'is-active', 'isc-dhcp-server'], capture_output=True, text=True).stdout.strip()
        status_ftp = subprocess.run(['systemctl', 'is-active', 'vsftpd'], capture_output=True, text=True).stdout.strip()
        status_ssh = subprocess.run(['systemctl', 'is-active', 'ssh'], capture_output=True, text=True).stdout.strip()
    except:
        status_dhcp = status_ftp = status_ssh = "inactive"
    return render_template('index.html', dhcp_vivo=status_dhcp, ftp_vivo=status_ftp, ssh_vivo=status_ssh)

@app.route('/servicios')
def ver_servicios():
    try:
        s_ssh = subprocess.run(['systemctl', 'is-active', 'ssh'], capture_output=True, text=True).stdout.strip()
        s_dhcp = subprocess.run(['systemctl', 'is-active', 'isc-dhcp-server'], capture_output=True, text=True).stdout.strip()
        s_ftp = subprocess.run(['systemctl', 'is-active', 'vsftpd'], capture_output=True, text=True).stdout.strip()
    except:
        s_ssh = s_dhcp = s_ftp = "inactive"
    return render_template('servicios.html', s_ssh=s_ssh, s_dhcp=s_dhcp, s_ftp=s_ftp)
```

Reinicios de servicios (`/reiniciar/<servicio>`):

Esta función sirve para reiniciar los servicios. Utiliza un diccionario para relacionar los nombres comunes de los servicios (`ssh`, `dhcp`, `ftp`) con los nombres técnicos de los servicios en el sistema operativo. La ejecución de los comandos se realiza con `subprocess.run` sin necesidad de privilegios de sudo (ya que no los necesitamos al estar previamente configurado en `sudoers`), permitiendo a los usuarios sin conocimientos de informática administrar los servicios sin usar el terminal.

```
@app.route('/reiniciar/<servicio>')
def reiniciar_servicio(servicio):
    nombres = {"ssh": "ssh", "dhcp": "isc-dhcp-server", "ftp": "vsftpd"}
    try:
        subprocess.run(['sudo', 'systemctl', 'restart', nombres.get(servicio)], check=True)
        return redirect(url_for('inicio'))
    except Exception as e: return str(e)
```

Configuración de DHCP (/configurar/dhcp):  
Esta función cumple dos funciones:

Lectura: Utiliza re para que no lea cosas innecesarias del archivo leyendo el archivo /etc/dhcp/dhcpd.conf y extraer los parámetros usados (subred, rango, router), mostrándolos en el formulario web.

Escritura atómica: Para aplicar cambios, el programa crea un archivo temporal en /tmp y lo mueve a su ubicación en la carpeta del servicio DHCP usando sudo mv. Con nos aseguramos que, si ocurre cualquier error mientras se configura el archivo, el archivo original nunca quede corrupto.

```
36 @app.route('/configurar/dhcp', methods=['GET', 'POST'])
37 def configurar_dhcp():
38     if request.method == 'GET':
39         config_actual = {"subred": "-", "mascara": "-", "inicio": "-", "fin": "-", "router": "-"}
40         try:
41             if os.path.exists('/etc/dhcp/dhcpd.conf'):
42                 with open('/etc/dhcp/dhcpd.conf', 'r') as f:
43                     content = f.read()
44                     sub = re.search(r"subnet ([\d.]+) netmask ([\d.]+)", content)
45                     if sub:
46                         config_actual["subred"] = sub.group(1)
47                         config_actual["mascara"] = sub.group(2)
48
49                     rng = re.search(r"range ([\d.]+) ([\d.]+)", content)
50                     if rng:
51                         config_actual["inicio"] = rng.group(1)
52                         config_actual["fin"] = rng.group(2)
53
54                     rt = re.search(r"option routers ([\d.]+)", content)
55                     if rt:
56                         config_actual["router"] = rt.group(1)
57         except Exception as e:
58             print(f"Error leyendo config: {e}")
59
60         return render_template('dhcp.html', c=config_actual)
61
62     subred = request.form['subred']
63     mascara = request.form['mascara']
64     rango_inicio = request.form['rango_inicio']
65
36 @app.route('/configurar/dhcp', methods=['GET', 'POST'])
37 def configurar_dhcp():
38     if request.method == 'GET':
39         config_actual = {"subred": "-", "mascara": "-", "inicio": "-", "fin": "-", "router": "-"}
40         try:
41             if os.path.exists('/etc/dhcp/dhcpd.conf'):
42                 with open('/etc/dhcp/dhcpd.conf', 'r') as f:
43                     content = f.read()
44                     sub = re.search(r"subnet ([\d.]+) netmask ([\d.]+)", content)
45                     if sub:
46                         config_actual["subred"] = sub.group(1)
47                         config_actual["mascara"] = sub.group(2)
48
49                     rng = re.search(r"range ([\d.]+) ([\d.]+)", content)
50                     if rng:
51                         config_actual["inicio"] = rng.group(1)
52                         config_actual["fin"] = rng.group(2)
53
54                     rt = re.search(r"option routers ([\d.]+)", content)
55                     if rt:
56                         config_actual["router"] = rt.group(1)
57         except Exception as e:
58             print(f"Error leyendo config: {e}")
59
60         return render_template('dhcp.html', c=config_actual)
61
62     subred = request.form['subred']
63     mascara = request.form['mascara']
64     rango_inicio = request.form['rango_inicio']
```

```

subred = request.form['subred']
mascara = request.form['mascara']
rango_inicio = request.form['rango_inicio']
rango_fin = request.form['rango_fin']
router = request.form['router']

nueva_configuracion = f"""
#Generado con NetManager
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet {subred} netmask {mascara} {{
    range {rango_inicio} {rango_fin};
    option routers {router};
    option domain-name-servers 8.8.8.8, 8.8.4.4;
}}
"""

try:
    ruta_tmp = "/tmp/dhcpd.conf.tmp"
    with open(ruta_tmp, 'w') as f: f.write(nueva_configuracion)
    subprocess.run(['sudo', 'mv', ruta_tmp, '/etc/dhcp/dhcpd.conf'], check=True)
    subprocess.run(['sudo', 'systemctl', 'restart', 'isc-dhcp-server'], check=True)
    return redirect(url_for('configurar_dhcp'))
except Exception as e:
    return f"Error: {e}"

```

Gestión usuarios con FTP (/crear-usuario-ftp, /eliminar-usuario-ftp):

Usa los comandos **useradd**, **chpasswd**, **userdel** para administrar usuarios del sistema. Se ha configurado el shell por defecto /usr/sbin/nologin para que los usuarios FTP no puedan usar comandos y solo acceder a sus carpetas, añadiendo más seguridad al sistema.

```

@app.route('/configurar/ftp')
def configurar_ftp():
    usuarios = [n for n in os.listdir('/home') if n != 'tfg']
    return render_template('ftp.html', usuarios=usuarios)

@app.route('/crear-usuario-ftp', methods=['POST'])
def crear_usuario_ftp():
    u, p = request.form['username'], request.form['password']
    try:
        subprocess.run(['sudo', 'useradd', '-m', '-s', '/usr/sbin/nologin', u], check=True)
        subprocess.run(['sudo', 'chpasswd'], input=f"{u}:{p}", text=True, check=True)
        return redirect(url_for('configurar_ftp'))
    except Exception as e: return str(e)

@app.route('/eliminar-usuario-ftp', methods=['POST'])
def eliminar_usuario_ftp():
    u = request.form['username']
    try:
        subprocess.run(['sudo', 'userdel', '-r', u], check=True)
        return redirect(url_for('configurar_ftp'))
    except Exception as e: return str(e)

```

### Diagnóstico de Logs (/estado/<servicio>):

Permite ver el estado de los servicios con **systemctl status**. El resultado del comando capturado con stdout se muestra en el navegador dentro de una etiqueta <pre>. Esto permite que el administrador o usuario vea el log puro del sistema sin tener que acceder remotamente al servidor, haciendo más simple las tareas de resolución de incidencias.

```
@app.route('/estado/<servicio>')
def ver_estado(servicio):
    nombres = {"ssh": "ssh", "dhcp": "isc-dhcp-server", "ftp": "vsftpd"}
    try:
        res = subprocess.run(['systemctl', 'status', nombres.get(servicio)], capture_output=True, text=True)
        resultado_consola = res.stdout
        pagina_logs = f"""
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="/static/style.css">
    <title>NetManager - Logs {servicio.upper()}</title>
</head>
<body>
    <div class="contenido-principal">
        <h1 class="titulo-seccion"> Registros del Sistema (Logs) </h1>
        <p class="subtitulo"> Inspección de eventos en tiempo real para: <strong>{servicio.upper()}</strong></p>
        <div class="seccion-ftp">
            <h2 class="titulo-modulo"> Salida del comando systemctl status </h2>
            <pre> {resultado_consola} </pre>
        </div>
        <br>
        <a href="/" class="boton-guardar"> Volver </a>
    </div>
</body>
</html>
"""
        return pagina_logs
    except Exception as e:
        return str(e)
```

### Inicialización y Apertura de Red (\_main\_)

Arranca el servidor web de la aplicación. Configurar el parámetro **host='0.0.0.0'** es indispensable para el TFG, ya que le ordena a Flask escuchar en todas las interfaces de red físicas y virtuales del servidor, lo que permite a todos los usuarios conectarse al panel introduciendo la IP del servidor desde sus ordenadores externos a través del puerto 9000. **debug=True** habilita el modo de depuración en caliente para desarrolladores.

```
234 if __name__ == '__main__':
235     app.run(host='0.0.0.0', port=9000, debug=True)
```

### Servicio de Inicio Automático:

Hemos añadido un servicio dentro de /etc/systemd/system para que el archivo app.py (el programa principal) se ejecute solo al reiniciar o encender el ordenador con el usuario tfg (no administrador). Además, este programa solo se va a ejecutar si el servidor esta conectado a la red

```
GNU nano 4.8 netmanager.service
[Unit]
Description=Servicio Flask para Netmanager
After=network.target

[Service]

User=tfg

WorkingDirectory /home/tfg/NetManager

ExecStart=/usr/bin/python3 /home/tfg/NetManager/app.py

Restart=always

[Install]
WantedBy=multi-user.target
```

## 6.4 Desarrollo del Frontend y diseño de la interfaz web

### 6.4.1 PLANTILLAS VISUALES (/templates)

Todas las pantallas del sistema están estructuradas de forma que heredan un diseño unificado: una barra de navegación lateral fija a la izquierda y el módulo operativo a la derecha.

#### A. Panel de Control (index.html)

Es la pantalla de bienvenida y el cuadro de mandos principal para el administrador de sistemas.

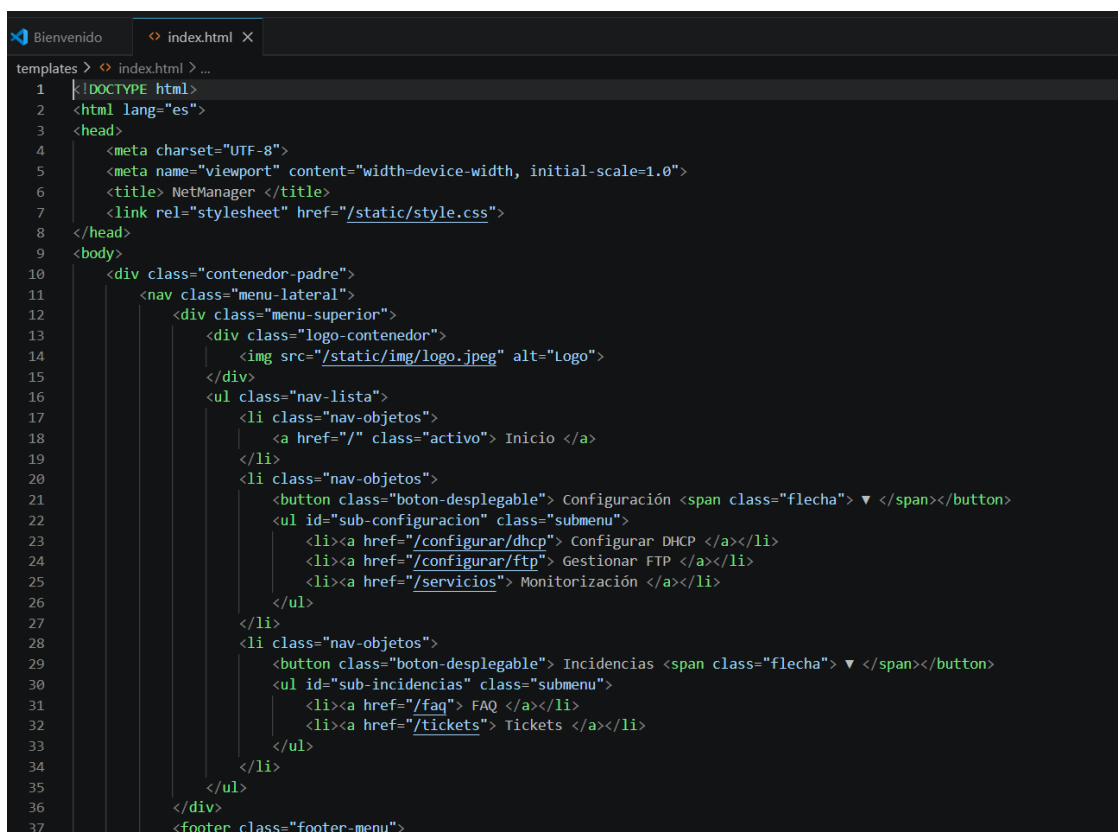
`<nav class="menu-lateral">` y `<ul>`: Delimita la barra vertical de la izquierda que contiene el menú principal.

`<main class="contenido-principal">`: Engloba de forma aislada todo el contenido útil de la derecha para evitar colisiones visuales con la barra lateral.

`<section class="panel-control">`: Estructura en formato de rejilla (conocido como Grid) las tres tarjetas informativas del sistema (SSH, DHCP y FTP).

`{% if ... %} / {{ ... if ... else ... }}` (Lógica Jinja2): Evalúa el estado del servicio enviado por Python en caliente. Si detecta la cadena "active", inyecta dinámicamente la clase CSS online junto al texto "En línea"; en caso contrario, activa la clase offline mostrando el texto "Apagado".

`<a href="/estado/ssh" class="boton-accion">`: Hipervínculos maquettados con estética de botón que invocan las rutas del servidor para auditar registros o reiniciar los demonios.



```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title> NetManager </title>
7   <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10   <div class="contenedor-padre">
11     <nav class="menu-lateral">
12       <div class="menu-superior">
13         <div class="logo-contenedor">
14           
15         </div>
16         <ul class="nav-lista">
17           <li class="nav-objetos">
18             <a href="/" class="activo"> Inicio </a>
19           </li>
20           <li class="nav-objetos">
21             <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22             <ul id="sub-configuracion" class="submenu">
23               <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24               <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25               <li><a href="/servicios"> Monitorización </a></li>
26             </ul>
27           </li>
28           <li class="nav-objetos">
29             <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30             <ul id="sub-incidencias" class="submenu">
31               <li><a href="/faq"> FAQ </a></li>
32               <li><a href="/tickets"> Tickets </a></li>
33             </ul>
34           </li>
35         </ul>
36       </div>
37       <div class="footer-menu">
```

```

37 <footer class="footer-menu">
38 <p class="nombre-autor"> Iker Carrizal </p>
39 <p class="nombre-autor"> Marcos García </p>
40 <p class="nombre-autor"> Marcos Padilla </p>
41 <div class="estado-sistema">
42 <span class="punto-online"></span> Sistema Conectado
43 </div>
44 </footer>
45 </nav>
46 <main class="contenido-principal">
47 <h1 class="titulo-seccion"> Panel de Control </h1>
48 <p class="subtitulo"> Monitorización del estado y administración de demonios locales en Linux. </p>
49 <section class="panel-control">
50 <div class="tarjeta">
51 <h3> Servicio SSH </h3>
52 <span class="estado {{ ssh_vivo }}"> {{ 'En línea' if ssh_vivo == 'active' else 'Apagado' }} </span><br><br>
53 <a href="/estado/ssh" class="boton-accion"> Logs </a>
54 <button class="boton-accion-peligro" onclick="abrirModal()"> Reiniciar </button>
55 </div>
56 <div class="tarjeta">
57 <h3> Servicio DHCP </h3>
58 <span class="estado {{ dhcp_vivo }}"> {{ 'En línea' if dhcp_vivo == 'active' else 'Apagado' }} </span><br><br>
59 <a href="/configurar/dhcp" class="boton-accion"> Gestionar </a>
60 <button class="boton-accion-peligro" onclick="abrirModal()"> Reiniciar </button>
61 </div>
62 <div class="tarjeta">
63 <h3> Servicio FTP </h3>
64 <span class="estado {{ ftp_vivo }}"> {{ 'En línea' if ftp_vivo == 'active' else 'Apagado' }} </span><br><br>
65 <a href="/configurar/ftp" class="boton-accion"> Gestionar </a>
66 <button class="boton-accion-peligro" onclick="abrirModal()"> Reiniciar </button>
67 </div>
68 </section>
69 </main>
70 </div>
71
72 <div id="ventana-confirmacion">
73 <div class="contenido-modal">
74 <h3> Confirmar Operación crítica</h3>
75 <p>¿Estás seguro de que deseas enviar esta instrucción al servidor? Los servicios dependientes podrían experimentar microcortes de conexión.</p>
76 <div class="botones-modal">
77 <button class="boton-guardar" onclick="confirmarAccion()">Sí, ejecutar</button>
78 <button class="boton-accion-peligro" onclick="cerrarModal()">Cancelar</button>
79 </div>
80 </div>
81 </div>
82
83 <script src="/static/main.js"></script>
84 </body>
85 </html>

```

## B. Configuración de Red (dhcp.html)

Permite visualizar los parámetros en ejecución del servidor DHCP y aplicar cambios en su fichero de configuración.

- `{{ c.subred }}` / `{{ c.mascara }}`: Marcadores de posición de Jinja2. El servidor Flask los sustituye automáticamente por los datos reales leídos previamente del archivo del sistema local.
- `<form action="/configurar/dhcp" method="POST">`: Formulario web que empaqueta y transmite la nueva configuración de red de manera protegida hacia el backend utilizando el método HTTP POST.
- `<input type="text" name="subred" required>`: Cuadros de texto para la introducción de las direcciones IP. El atributo name es indispensable para que Python identifique el dato, y required actúa como validación en el cliente impidiendo envíos en blanco.
- `<button type="submit">`: Elemento desencadenante que fuerza el envío de los parámetros hacia el script del servidor.

```
dhcp.html X
templates > dhcp.html > ...
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title> NetManager - DHCP </title>
7   <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10   <div class="contenedor-padre">
11     <nav class="menu-lateral">
12       <div class="menu-superior">
13         <div class="logo-contenedor">
14           
15         </div>
16         <ul class="nav-lista">
17           <li class="nav-objetos">
18             <a href="/"> Inicio </a>
19           </li>
20           <li class="nav-objetos">
21             <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22             <ul id="sub-configuracion" class="submenu">
23               <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24               <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25               <li><a href="/servicios"> Monitorización </a></li>
26             </ul>
27           </li>
28           <li class="nav-objetos">
29             <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30             <ul id="sub-incidencias" class="submenu">
31               <li><a href="/faq"> FAQ </a></li>
32               <li><a href="/tickets"> Tickets </a></li>
33             </ul>
34           </li>
35         </ul>
36       </div>
37       <div class="Footer-menu">
38         <p class="nombre-autor"> Iker Carrizal </p>
39         <p class="nombre-autor"> Marcos García </p>
40         <p class="nombre-autor"> Marcos Padilla </p>
41         <div class="estado-sistema">
42           <span class="punto-online"></span> Sistema Conectado
43         </div>
44       </div>
45     </nav>
46     <main class="contenido-principal">
47       <h1 class="titulo-seccion"> Configuración de Red DHCP </h1>
48       <p class="subtitulo"> Gestión de direccionamiento dinámico para clientes locales </p>
49       <div class="seccion-ftp">
50         <h2 class="titulo-modulo titulo-actual"> Configuración Actual </h2>
51         <div class="contenedor-tabla tabla-margen-dhcp">
52           <table class="tabla-servicios">
53             <thead>
54               <tr>
```

```

55         <tr>
56             <th> Parámetro </th>
57             <th> Valor Actual </th>
58         </tr>
59     </thead>
60     <tbody>
61         <tr>
62             <td><strong> Red / Máscara </strong></td>
63             <td> {{ c.subred }} / {{ c.mascara }} </td>
64         </tr>
65         <tr>
66             <td><strong> Rango IP </strong></td>
67             <td> {{ c.inicio }} - {{ c.fin }} </td>
68         </tr>
69         <tr>
70             <td><strong> Puerta de Enlace </strong></td>
71             <td> {{ c.router }} </td>
72         </tr>
73     </tbody>
74 </table>
75 </div>
76 </div>
77 <hr>
78 <div class="seccion-ftp">
79     <h2 class="titulo-modulo"> Parámetros de Subred </h2>
80     <div class="contenedor-formulario">
81         <form action="/configurar/dhcp" method="POST">
82             <div class="formulario-fila-grid">
83                 <div class="grupo-input">
84                     <label> Dirección de Subred: </label>
85                     <input type="text" name="subred" placeholder="192.168.1.0" required>
86                 </div>
87                 <div class="grupo-input">
88                     <label> Máscara de Red: </label>
89                     <input type="text" name="mascara" placeholder="255.255.255.0" required>
90                 </div>
91             </div>
92             <hr class="divisor-formulario-interno">
93             <h2 class="titulo-modulo titulo-rango"> Rango de Asignación </h2>
94             <div class="formulario-fila-grid">
95                 <div class="grupo-input">
96                     <label> IP Inicio (Range Start): </label>
97                     <input type="text" name="rango_inicio" placeholder="192.168.1.100" required>
98                 </div>
99                 <div class="grupo-input">
100                     <label> IP Fin (Range End): </label>
101                     <input type="text" name="rango_fin" placeholder="192.168.1.200" required>
102                 </div>
103             </div>
104             <div class="grupo-input">
105                 <div class="grupo-input">
106                     <label> Puerta de Enlace (Gateway): </label>
107                     <input type="text" name="router" placeholder="192.168.1.1" required>
108                 </div>
109                 <div class="caja-aviso-advertencia">
110                     <strong> Nota:</strong> Al aplicar los cambios, el servicio isc-dhcp-server se reiniciará automáticamente. Asegúrate de que los rangos no entren en conflicto con IPs estáticas.
111                 </div>
112                 <button type="submit" class="boton-guardar"> Aplicar Configuración y Reiniciar Servicio </button>
113             </div>
114         </form>
115     </div>
116 </main>
117 <script src="/static/main.js"></script>
118 </body>
119 </html>

```



## C. Administración FTP ([ftp.html](#))

Módulo encargado de interactuar con el árbol de cuentas de usuario del sistema operativo destinadas al servidor de archivos compartidos.

- Dos etiquetas `<form>` independientes: Separan de forma estricta la operación de "Alta de Usuario" (apunta a la ruta `/crear-usuario-ftp`) de la operación de "Baja de Usuario" (vinculada a `/eliminar-usuario-ftp`).

- `<input type="password">`: Campo de seguridad que enmascara los caracteres tecleados transformándolos en puntos para salvaguardar la privacidad de las credenciales.

- `{% for u in usuarios %}` (Bucle iterativo de Jinja2): Estructura cíclica que recorre la lista de usuarios Linux remitida por Python. Por cada cuenta existente en el sistema, genera de forma automatizada una fila `<tr><td>` dentro de la tabla de visualización.

- `<span class="punto-verde"></span>`: Etiqueta contenedora vacía que CSS estiliza en forma de indicador esférico verde para reflejar que la cuenta está plenamente operativa.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title> NetManager - FTP </title>
7   <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10  <div class="contenedor-padre">
11    <nav class="menu-lateral">
12      <div class="menu-superior">
13        <div class="logo-contenedor">
14          
15        </div>
16        <ul class="nav-lista">
17          <li class="nav-objetos">
18            <a href="/"> Inicio </a>
19          </li>
20          <li class="nav-objetos">
21            <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22            <ul id="sub-configuracion" class="submenu">
23              <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24              <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25              <li><a href="/servicios"> Monitorización </a></li>
26            </ul>
27          </li>
28          <li class="nav-objetos">
29            <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30            <ul id="sub-incidencias" class="submenu">
31              <li><a href="/faq"> FAQ </a></li>
32              <li><a href="/tickets"> Tickets </a></li>
33            </ul>
34          </li>
35        </ul>
36      </div>
37      <div class="footer-menu">
38        <p class="nombre-autor"> Iker Carrizal </p>
39        <p class="nombre-autor"> Marcos García </p>
40        <p class="nombre-autor"> Marcos Padilla </p>
41        <div class="estado-sistema">
42          <span class="punto-online"></span> Sistema Conectado
43        </div>
44      </div>
45    </nav>
46    <div class="contenido-principal">
47      <h1 class="titulo-seccion"> Administración FTP </h1>
48      <p class="subtitulo"> Gestión de cuentas locales de almacenamiento seguro y control de accesos vsftpd. </p>
49
50      <div class="seccion-ftp">
51        <h2 class="titulo-modulo"> Nuevo Usuario </h2>
52        <div class="contenedor-formulario">
53          <form action="/crear-usuario-ftp" method="POST">
54            <div class="grupo-input">
55              <label> Nombre: </label>
```

```

ftp.html x
templates > ftp.html > ...
2 <html lang="es">
9 <body>
10 <div class="contenedor-padre">
46 <main class="contenido-principal">
50 <div class="seccion-ftp">
52 <div class="contenedor-formulario">
53 <form action="/crear-usuario-ftp" method="POST">
54 <div class="grupo-input">
55 <label> Nombre: </label>
56 <input type="text" name="username" required>
57 </div>
58 <div class="grupo-input">
59 <label> Contraseña: </label>
60 <input type="password" name="password" required>
61 </div>
62 <button type="submit" class="boton-guardar"> Dar de Alta </button>
63 </form>
64 </div>
65 </div>
66 <hr>
67 <div class="seccion-ftp">
68 <h2 class="titulo-modulo"> Baja de Usuario </h2>
69 <div class="contenedor-formulario">
70 <form action="/eliminar-usuario-ftp" method="POST">
71 <div class="grupo-input">
72 <label> Nombre de usuario a borrar: </label>
73 <input type="text" name="username" required>
74 </div>
75 <button type="submit" class="boton-accion-peligro"> Borrar permanentemente </button>
76 </form>
77 </div>
78 </div>
79 <hr>
80 <div class="seccion-ftp">
81 <h2 class="titulo-modulo"> Usuarios Activos </h2>
82 <div class="contenedor-tabla">
83 <table class="tabla-servicios">
84 <thead>
85 <tr>
86 <th> Usuario </th>
87 <th> Ruta </th>
88 <th> Estado </th>
89 </tr>
90 </thead>
91 <tbody>
92 <tr>
93 <td><strong> {{ u }} </strong></td>
94 <td> /home/{{ u }} </td>
95 <td>
96 <span class="punto-verde"></span> Activo
97 </td>
98 </tr>
99 <tr>
100 <td> {% for u in usuarios %}
101 <td> {{ u }}
102 <td> /home/{{ u }}
103 <td>
104 <span class="punto-verde"></span> Activo
105 </td>
106 </tr>
107 <tr>
108 <td> {% endfor %}
109 </td>
110 </tr>
111 </tbody>
112 </table>
113 </div>
114 </div>
115 </main>
116 </div>
117 <script src="/static/main.js"></script>
118 </body>
119 </html>

```

## D. Monitorización de Servicios (servicios.html)

Módulo orientado a la auditoría técnica elemental del estado de los sockets del servidor.

- `<table class="tabla-servicios">`: Genera un contenedor matricial organizado para clasificar datos de red de forma estructurada.
- `<thead>` y `<tbody>`: Segmentan la tabla aislando los títulos de cabecera (Servicio, Puerto, Estado) de los registros dinámicos correspondientes al cuerpo de datos.
- `<tr>`, `<th>` y `<td>`: `<tr>` define una fila horizontal completa; `<th>` asigna una celda de título con formato destacado; `<td>` representa una celda convencional de información. Muestra de forma estática los puertos de red normativos (22, 67 y 21).

```

servicios.html X
templates > > servicios.html > ...
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title> NetManager - Servicios </title>
7      <link rel="stylesheet" href="/static/style.css">
8  </head>
9  <body>
10     <div class="contenedor-padre">
11         <nav class="menu-lateral">
12             <div class="menu-superior">
13                 <div class="logo-contenedor">
14                     
15                 </div>
16                 <ul class="nav-lista">
17                     <li class="nav-objetos">
18                         <a href="/"> Inicio </a>
19                     </li>
20                     <li class="nav-objetos">
21                         <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22                         <ul id="sub-configuracion" class="submenu">
23                             <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24                             <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25                             <li><a href="/servicios"> Monitorización </a></li>
26                         </ul>
27                     </li>
28                     <li class="nav-objetos">
29                         <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30                         <ul id="sub-incidencias" class="submenu">
31                             <li><a href="/faq"> FAQ </a></li>
32                             <li><a href="/tickets"> Tickets </a></li>
33                         </ul>
34                     </li>
35                 </ul>
36             </div>
37             <div class="footer-menu">
38                 <p class="nombre-autor"> Iker Carrizal </p>
39                 <p class="nombre-autor"> Marcos García </p>
40                 <p class="nombre-autor"> Marcos Padilla </p>
41                 <div class="estado-sistema">
42                     <span class="punto-online"></span> Sistema Conectado
43                 </div>
44             </div>
45         </nav>
46         <main class="contenido-principal">
47             <h1 class="titulo-seccion"> Monitorización de Demonios </h1>
48             <p class="subtitulo"> Comprobación del estado operacional de los principales sockets de red de la empresa. </p>
49             <div class="seccion-ftp">
50                 <h2 class="titulo-modulo"> Sockets Escuchando </h2>
51                 <div class="contenedor-tabla">
52                     <table class="tabla-servicios">
53                         <thead>
54                             <tr>

```

```

55         <tr>
56             <th> Servicio Linux </th>
57             <th> Puerto Base </th>
58             <th> Estado Kernel </th>
59         </tr>
60     </thead>
61     <tbody>
62         <tr>
63             <td><strong> SSH Server (OpenSSH) </strong></td>
64             <td> 22 </td>
65             <td><span class="estado {{ s_ssh }}"> {{ s_ssh }} </span></td>
66         </tr>
67         <tr>
68             <td><strong> ISC DHCP Server </strong></td>
69             <td> 67 </td>
70             <td><span class="estado {{ s_dhcp }}"> {{ s_dhcp }} </span></td>
71         </tr>
72         <tr>
73             <td><strong> VSFTPD Daemon </strong></td>
74             <td> 21 </td>
75             <td><span class="estado {{ s_ftp }}"> {{ s_ftp }} </span></td>
76         </tr>
77     </tbody>
78 </table>
79 </div>
80 </div>
81 </main>
82 </div>
83 <script src="/static/main.js"></script>
84 </body>
85 </html>

```

## E. Preguntas Frecuentes (faq.html)

Manual de resolución de problemas automatizado cuyo fin es dotar de autonomía a los usuarios de la Pyme y mitigar incidencias redundantes.

- `<li class="item-desplegable">`: Nodo de lista que empaqueta el botón maestro y el submenú interno. Es la pieza clave sobre la que actúa el ratón para ejecutar el efecto de despliegue por CSS.
- `<button class="boton-desplegable">`: Botón textual que muestra la categoría jerárquica (Configuración o Incidencias) junto a una flecha de dirección (▼).
- `<ul id="..." class="submenu">`: Lista anidada que contiene las opciones ocultas de navegación secundaria. Por defecto permanece invisible.
- `<div class="bloque-faq">`: Agrupa de manera aislada cada pregunta técnica (`<h3>`) con su párrafo aclaratorio de solución (`<p>`) para dotar al módulo de márgenes y líneas de separación limpias.

```
faq.html X
templates > faq.html > ...
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title> NetManager - Preguntas Frecuentes </title>
7   <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10   <div class="contenedor-padre">
11     <nav class="menu-lateral">
12       <div class="menu-superior">
13         <div class="logo-contenedor">
14           
15         </div>
16         <ul class="nav-lista">
17           <li class="nav-objetos">
18             <a href="/"> Inicio </a>
19           </li>
20           <li class="nav-objetos">
21             <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22             <ul id="sub-configuracion" class="submenu">
23               <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24               <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25               <li><a href="/servicios"> Monitorización </a></li>
26             </ul>
27           </li>
28           <li class="nav-objetos">
29             <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30             <ul id="sub-incidencias" class="submenu">
31               <li><a href="/faq"> FAQ </a></li>
32               <li><a href="/tickets"> Tickets </a></li>
33             </ul>
34           </li>
35         </ul>
36       </div>
37       <div class="footer-menu">
38         <p class="nombre-autor"> Iker Carrizal </p>
39         <p class="nombre-autor"> Marcos García </p>
40         <p class="nombre-autor"> Marcos Padilla </p>
41         <div class="estado-sistema">
42           <span class="punto-online"></span> Sistema Conectado
43         </div>
44       </div>
45     </nav>
46     <main class="contenido-principal">
47       <h1 class="titulo-seccion"> Preguntas Frecuentes (FAQ) </h1>
48       <hr>
49       <p class="subtitulo"> Soluciones rápidas a los problemas más comunes en la red de la Pyme. </p>
50       <div>
51         <div class="contenedor-formulario">
52           <div class="bloque-faq">
53             <h3> 1. ¿Mi ordenador no obtiene dirección IP? </h3>
54             <p> Verifica que el servicio <strong>DHCP </strong> esté iniciado en el servidor. Si el servidor está caído, los clientes no podrán navegar ni conectarse a los recursos internos. </p>
```

```
55           </div>
56           <div class="bloque-faq">
57             <h3> 2. ¿No puedo subir archivos al FTP? </h3>
58             <p> Comprueba los permisos de escritura del usuario. Recuerda que en <strong>FTP </strong> no solo cuenta el permiso del servicio, sino también el privilegio del sistema de archivos local en L
59           </div>
60           <div class="bloque-faq">
61             <h3> 3. ¿Por qué recibo un error de "Connection Refused" al usar SSH? </h3>
62             <p> Esto ocurre generalmente cuando el servicio <strong>SSH </strong> está detenido en el servidor Linux o el puerto 22 está siendo bloqueado por el firewall local (ufw). </p>
63           </div>
64           <div class="bloque-faq">
65             <h3> 4. ¿Qué ocurre si dos equipos tienen la misma IP estática en la red? </h3>
66             <p> Se genera un conflicto de direcciones IP que causará pérdida de paquetes e intermitencia en ambos clientes. Asegúrate de configurar en el módulo <strong>DHCP </strong> un rango de asignaci
67           </div>
68           <div class="bloque-faq">
69             <h3> 5. ¿Los cambios aplicados en los formularios se borran al apagar el servidor? </h3>
70             <p> No. Al pulsar los botones de guardar, los scripts de la aplicación modifican directamente los archivos de configuración en caliente en Linux (como el archivo del pool de subredes del servic
71           </div>
72           <div class="bloque-faq">
73             <h3> 6. ¿Qué diferencia hay entre la tarjeta de estado del Inicio y la Monitorización de Servicios? </h3>
74             <p> El panel de inicio monitoriza si los demonios base están cargados en la memoria del sistema, mientras que el módulo de <strong>Monitorización de Servicios </strong> comprueba específicamen
75           </div>
76         </div>
77       </main>
78     </div>
79     <script src="/static/main.js"></script>
80 </body>
81 </html>
```

## F. Soporte Técnico (tickets.html)

Interfaz interactiva que permite a los empleados del entorno local reportar averías técnicas de su puesto de trabajo.

- `<select class="selector-incidencia" id="ticket-tipo">`: Menú desplegable que restringe la entrada de datos del usuario, forzándolo a escoger categorías predefinidas (DHCP, FTP u Otros) por medio de las subetiquetas `<option>`.
- `<textarea class="descripcion-ticket">`: Área de texto multilínea que ofrece un espacio amplio para detallar la naturaleza de la avería observada en el hardware o en la conectividad.
- `<button ... onclick="enviarTicket()>`: Elemento de acción asociado a un evento de clic directo (onclick) que transfiere la información hacia la lógica de JavaScript sin necesidad de recargar el navegador.

```
templates > tickets.html > ...
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title> NetManager - Tickets </title>
7      <link rel="stylesheet" href="/static/style.css">
8  </head>
9  <body>
10     <div class="contenedor-padre">
11         <nav class="menu-lateral">
12             <div class="menu-superior">
13                 <div class="logo-contenedor">
14                     
15                 </div>
16                 <ul class="nav-lista">
17                     <li class="nav-objetos">
18                         <a href="/"> Inicio </a>
19                     </li>
20                     <li class="nav-objetos">
21                         <button class="boton-desplegable"> Configuración <span class="flecha"> ▼ </span></button>
22                         <ul id="sub-configuracion" class="submenu">
23                             <li><a href="/configurar/dhcp"> Configurar DHCP </a></li>
24                             <li><a href="/configurar/ftp"> Gestionar FTP </a></li>
25                             <li><a href="/servicios"> Monitorización </a></li>
26                         </ul>
27                     </li>
28                     <li class="nav-objetos">
29                         <button class="boton-desplegable"> Incidencias <span class="flecha"> ▼ </span></button>
30                         <ul id="sub-incidencias" class="submenu">
31                             <li><a href="/faq"> FAQ </a></li>
32                             <li><a href="/tickets"> Tickets </a></li>
33                         </ul>
34                     </li>
35                 </ul>
36             </div>
37             <div class="footer-menu">
38                 <p class="nombre-autor"> Iker Carrizal </p>
39                 <p class="nombre-autor"> Marcos García </p>
40                 <p class="nombre-autor"> Marcos Padilla </p>
41                 <div class="estado-sistema">
42                     <span class="punto-online"></span> Sistema Conectado
43                 </div>
44             </div>
45         </nav>
46         <main class="contenido-principal">
47             <h1 class="titulo-seccion titulo-soporte-ticket"> Soporte Técnico - Crear Ticket </h1>
48             <hr>
49             <br>
50             <section class="contenedor-formulario">
51                 <form id="form-ticket">
52                     <div class="grupo-input">
53                         <label> Nombre del Usuario: </label>
54                         <input type="text" id="ticket-usuario" placeholder="Ej: Juan Pérez" required>
55                     </div>
56                 </form>
57             </section>
58         </main>
59     </div>
60 </body>
61 </html>
```

```

55 </div>
56 <div class="grupo-input">
57   <label> Tipo de Incidencia: </label>
58   <select class="selector-incidencia" id="ticket-tipo">
59     <option> Problema con DHCP </option>
60     <option> Problema con FTP </option>
61     <option> Otros </option>
62   </select>
63 </div>
64 <div class="grupo-input">
65   <label> Descripción: </label>
66   <textarea class="descripcion-ticket" id="ticket-descripcion" rows="5" placeholder="Explica brevemente qué sucede..."></textarea>
67 </div>
68 <button type="button" class="boton-guardar" onclick="enviarTicket()"> Enviar Ticket </button>
69 </form>
70 </section>
71 </main>
72 </div>
73 <script src="/static/main.js"></script>
74 </body>
75 </html>

```

### 6.4.2 CAPA DE DISEÑO (static/style.css)

Hoja de estilos encargada de unificar la identidad gráfica de la aplicación web a través de selectores de etiquetas y clases (.).

- `* { box-sizing: border-box; }`: El selector universal `*` inicializa los márgenes y rellenos internos. La propiedad `border-box` garantiza que las dimensiones declaradas en un bloque incluyan de forma exacta el grosor de sus bordes y rellenos, evitando desajustes visuales.
- `body { display: flex; overflow: hidden; }`: Define la ventana del navegador como un contenedor flexible horizontal. La propiedad `overflow: hidden` deshabilita las barras de scroll dobles generales de la pantalla, fijando la interfaz para un uso fluido.
- `.menu-lateral { height: 100%; min-width: 260px; }`: Obliga a la barra lateral de color oscuro (`#1a1a1a`) a extenderse a lo largo del 100% de la altura de la ventana y bloquea un ancho de seguridad para salvaguardar la legibilidad de la navegación en pantallas reducidas.
- `.tarjeta: hover { transform: translateY(-5px); }`: Transición dinámica intermedia. Al posicionar el cursor sobre una tarjeta de servicio, el elemento se desplaza verticalmente hacia arriba 5 píxeles, simulando un efecto de relieve e interacción tridimensional.
- `.item-desplegable: hover .submenu { display: block !important; }`: Mecanismo de Despliegue de Menús: Establece el comportamiento interactivo del sidebar mediante CSS puro. Los submenús permanecen ocultos bajo la propiedad `display: none`. Cuando el navegador detecta el posicionamiento del ratón (`:hover`) sobre la clase contenedora, conmuta el estado de manera automática a `display: block`, provocando que el menú emerja al entrar y se guarde al salir instantáneamente.

### 6.4.3 INTERACTIVIDAD DEL PROGRAMA (static/main.js)

Lógica de scripts ejecutada directamente en el lado del cliente (navegador web del usuario).

- `document.getElementById("ventana-confirmacion")`: Función encargada de examinar el árbol del documento HTML para localizar e instanciar en memoria el nodo exacto que posee dicho identificador único (la ventana modal).
- `function abrirModal() / cerrarModal()`: Rutas algorítmicas encargadas de alternar dinámicamente las reglas CSS de visibilidad (`style.display = "flex" o "none"`) para mostrar u ocultar la ventana modal de confirmación antes de ejecutar operaciones críticas del sistema.
- `window.onclick = function(event) { ... }`: Escuchador de eventos global incorporado para auditar los clics efectuados en pantalla. Si detecta que el usuario pincha fuera de la caja operativa (sobre el fondo oscuro traslúcido `event.target == modalConfirmacion`), ejecuta de forma automatizada la rutina de cierre para maximizar la usabilidad del panel.

```
static > JS mainjs > ...
1  const modalConfirmacion = document.getElementById("ventana-confirmacion");
2
3  function abrirModal() {
4      modalConfirmacion.style.display = "flex";
5      console.log("Sistema: El usuario ha solicitado una gestión de servicio.");
6  }
7
8  function cerrarModal() {
9      modalConfirmacion.style.display = "none";
10 }
11
12 function confirmarAccion() {
13     alert("¡Orden enviada! El servicio se está procesando en el servidor.");
14     cerrarModal();
15 }
16
17 window.onclick = function(event) {
18     if (event.target == modalConfirmacion) {
19         cerrarModal();
20     }
21 }
22
23 // Esta función se activa cuando pulsas el botón del HTML
24 function enviarTicket() {
25     // 1. Cogemos el texto que hay dentro de cada caja del HTML
26     var elUsuario = document.getElementById("ticket-usuario").value;
27     var elTipo = document.getElementById("ticket-tipo").value;
28     var laDescripcion = document.getElementById("ticket-descripcion").value;
29
30     // 2. Si alguna caja está vacía, regañamos al usuario y no hacemos nada más
31     if (elUsuario == "" || laDescripcion == "") {
32         alert("Tienes que rellenar todos los campos.");
33         return;
34     }
35
36     // 3. Juntamos los datos en una mochila para el cartero (FormData es lo más fácil)
37     var mochila = new FormData();
38     mochila.append("usuario_html", elUsuario);
39     mochila.append("tipo_html", elTipo);
40     mochila.append("descripcion_html", laDescripcion);
41
42     // 4. Mandamos al cartero corriendo hacia la ruta de Python llamada '/guardar-ticket'
43     fetch('/guardar-ticket', {
44         method: 'POST',
45         body: mochila
46     })
47     .then(function(respuesta) {
48         // Cuando llega el cartero de vuelta, comprobamos que todo esté OK
49         if (respuesta.ok) {
50             alert("El ticket se ha guardado en el servidor.");
51             // Vaciamos el formulario para que se quede limpio
52             document.getElementById("form-ticket").reset();
53         } else {
54             alert("El servidor ha dado un error.");
55         }
56     })
57 }
```

## 6.5.- Funcionalidades adicionales y módulos de soporte

### A. Controladores de Rutas y Enrutamiento Estático (/faq y /tickets)

Los decoradores `@app.route` actúan como carteles indicadores. Le marcan a Flask qué función debe ejecutarse cuando el usuario escribe una URL en la web. La instrucción `render_template` es la encargada de buscar en el directorio `/templates` el archivo HTML indicado para servírselo limpio y renderizado al cliente.

```
111 @app.route('/faq')
112 def ver_faq():
113     return render_template('faq.html')
114
115 @app.route('/tickets')
116 def ver_tickets():
117     return render_template('tickets.html')
118
```



## B. Módulo de Almacenamiento y Persistencia de Incidencias (/guardar-ticket)

Procesa la información enviada mediante el método seguro HTTP POST desde el formulario de soporte técnico.

El objeto `request.form` captura las cadenas escritas por el usuario. Posteriormente, `texto_final` estructura los datos en una sola línea formateada con barras divisorias (|) y un salto de línea (`\n`). La instrucción `with open(..., "a")` abre el fichero local en modo adjuntar (Append), lo que garantiza escribir la nueva avería abajo del todo en el bloc de notas sin sobrescribir las incidencias existentes. El bloque `try/except` actúa como salvavidas evitando el bloqueo de la web si el fichero carece de permisos de escritura en el sistema Linux.

```
@app.route('/guardar-ticket', methods=['POST'])
def guardar_ticket_en_servidor():
    try:
        quien = request.form['usuario_html']
        que_pasa = request.form['tipo_html']
        detalle = request.form['descripcion_html']
        texto_final = f"USUARIO: {quien} | INCIDENCIA: {que_pasa} | DETALLE: {detalle}\n"
        with open("tickets_soporte.txt", "a", encoding="utf-8") as archivo:
            archivo.write(texto_final)
        return "Hecho"
    except Exception as e:
        return str(e), 500
```

## C. Procesamiento Dinámico de Datos e Inyección HTML (/ver-tickets)

Genera una página web dinámica directamente desde Python para visualizar el historial.

Lee el archivo en modo "r" (Read, lectura) y guarda las líneas en una lista. Mediante un bucle `for`, analiza cada fila aplicando el método `.split("|")` para segmentar la cadena de texto en tres partes indexadas (Usuario, Avería y Descripción). Tras limpiar las etiquetas innecesarias con `.replace()`, la script maqueta de forma iterativa las filas de la tabla (`<tr><td>`) y las incrusta dentro del código HTML general que se le envía al navegador. Si el archivo no existe (bloque `except`), el sistema lo gestiona mostrando un aviso controlado de base de datos vacía.

```

132 @app.route('/ver-tickets')
133 def ver_tickets_guardados():
134     try:
135         with open("tickets_soporte.txt", "r", encoding="utf-8") as archivo:
136             todas_las_lineas = archivo.readlines()
137             filas_de_la_tabla = ""
138             for una_linea in todas_las_lineas:
139                 if una_linea.strip() != "":
140                     partes = una_linea.split("|")
141                     nombre_limpio = partes[0].replace("USUARIO:", "").strip()
142                     tipo_limpio = partes[1].replace("INCIDENCIA:", "").strip()
143                     detalle_limpio = partes[2].replace("DETALLE:", "").strip()
144                     filas_de_la_tabla += f"""
145                     <tr>
146                         <td><strong> {nombre_limpio} </strong></td>
147                         <td><span class="estado active"> {tipo_limpio} </span></td>
148                         <td> {detalle_limpio} </td>
149                     </tr>
150                     """
151             pagina_completa = f"""
152             <!DOCTYPE html>
153             <html lang="es">
154             <head>
155                 <meta charset="UTF-8">
156                 <title> NetManager - Historial </title>
157                 <link rel="stylesheet" href="/static/style.css">
158             </head>
159             <body>
160                 <div class="contenido-principal">
161                     <h1 class="titulo-seccion"> Historial de Incidencias </h1>
162                     <p class="subtitulo"> Listado de problemas técnicos guardados en el servidor. </p>
163                     <div class="seccion-ftp">
164                         <h2 class="titulo-modulo"> Tickets Registrados </h2>
165                         <div class="contenedor-tabla">
166                             <table class="tabla-servicios">
167                                 <thead>
168                                     <tr>
169                                         <th> Usuario </th>
170                                         <th> Avería </th>
171                                         <th> Descripción </th>
172                                     </tr>
173                                 </thead>
174                                 <tbody>
175                                     <tbody>
176                                         {filas_de_la_tabla}
177                                     </tbody>
178                                 </tbody>
179                             </table>
180                         </div>
181                     </div>
182                     <br>
183                     <a href="/" class="boton-guardar"> Volver </a>
184                 </div>
185             </body>
186             </html>
187             """
188             return pagina_completa
189         except Exception as e:
190             return """
191             <!DOCTYPE html>
192             <html>
193             <head><link rel="stylesheet" href="/static/style.css"></head>
194             <body>
195                 <div class="contenido-principal">
196                     <h2 class="titulo-seccion"> Aún no hay ningún ticket registrado </h2>
197                     <p class="subtitulo"> El bloc de notas del servidor está limpio. </p>
198                     <br>
199                     <a href="/" class="boton-guardar"> Volver </a>
200                 </div>
201             </body>
202             </html>
203             """

```

## 7.- Planificación del proyecto

---

### 7.1.- Acciones

Para el desarrollo de NetManager de forma estructurada y cumplir con los plazos de entrega, el trabajo lo dividimos en tareas específicas que incluye desde la administración de sistemas hasta el desarrollo web. Estas acciones las hemos organizado en las siguientes fases principales:

#### 1) Fase de Infraestructura y Sistemas Base (Análisis y Diseño)

- Creación y configuración del entorno de pruebas mediante hipervisores (VirtualBox), desplegando la máquina virtual servidor (Ubuntu Server) y las máquinas cliente.
- Diseño topológico y configuración de las interfaces de red estáticas y dinámicas (Adaptador Puente y Red Interna).
- Instalación en el servidor de los demonios y servicios base a automatizar: isc-dhcp-server, vsftpd y openssh-server.

#### 2) Fase de Backend y Control de Permisos (Core en Flask y DHCP)

- Desarrollo del servidor de aplicaciones utilizando Python y el framework Flask, configurado para escuchar en el puerto 9000 (0.0.0.0).
- Programación de los módulos lógicos (mediante la librería subprocess) para modificar los archivos de configuración del sistema (como /etc/dhcp/dhcpd.conf) de forma atómica y segura.
- Configuración avanzada de seguridad en el archivo /etc/sudoers para permitir al backend ejecutar comandos críticos (systemctl restart, useradd, chpasswd) aplicando el principio de menor privilegio.

#### 3) Fase de Frontend e Interfaz de Usuario (Servicio FTP y pulido de UI)

- Diseño y maquetación de la interfaz gráfica del panel de control utilizando HTML, CSS y JavaScript.
- Desarrollo de las plantillas visuales unificadas (Dashboard de inicio, panel de monitorización, gestión de usuarios FTP y configuración DHCP).
- Implementación de módulos interactivos de soporte, incluyendo el manual de Preguntas Frecuentes (FAQ) y el sistema automatizado de registro de tickets de incidencias.

#### 4) Fase de Empaquetado, Pruebas y Despliegue

- Ejecución de las pruebas de conectividad y auditoría de creación de usuarios FTP y asignación de IPs.
- Configuración del servicio systemd local (netmanager.service) para el arranque automático de la plataforma con el inicio del servidor.
- Creación de la imagen ISO personalizada y desatendida del servidor para facilitar su instalación en entornos de PYME.
- Redacción de la documentación técnica y elaboración de la memoria del proyecto.

## 7.2.- Temporalización y secuenciación

| Mes   | Semana | Tareas                                                                                | Hito al que pertenece cada tarea |
|-------|--------|---------------------------------------------------------------------------------------|----------------------------------|
| Marzo | S1     | <b>Inicio:</b> Reunión de equipo, definición de nuestro proyecto, ISO Appliance.      | Hito 1                           |
|       | S2     | <b>Análisis:</b> Reparto de módulos (Iker, Marcos G., Padilla), diseño lógico/físico. | Hito 1                           |
|       | S3     | <b>Entorno:</b> Instalación de Ubuntu Server focal y VS Code.                         | Hito 1                           |
|       | S4     | <b>Entrega Hito 1:</b> Documentación inicial, creación de preguntas/respuestas.       | <b>Hito 1 (26/03)</b>            |
| Abril | S1     | <b>Core Flask:</b> Programación de la estructura del app.py, rutas básicas.           | Hito 2                           |
|       | S2     | <b>Módulo DHCP:</b> Lógica de subprocess para /etc/dhcpd.conf, permisos sudoers.      | Hito 2                           |
|       | S3     | <b>UI Hito 2:</b> Diseño de la primera versión del HTML/CSS e integración.            | Hito 2                           |
|       | S4     | <b>Entrega Hito 2:</b> Grabación del vídeo (musitado), pulido técnico final.          | <b>Hito 2 (20/04)</b>            |
| Mayo  | S1     | <b>Módulo FTP:</b> Instalación/configuración de vsftpd, gestión de usuarios.          | Hito 3                           |
|       | S2     | <b>Depuración Extrema:</b> Resolución del DDoS en repositorios de Ubuntu, DNS.        | Hito 3                           |
|       | S3     | <b>Revisión: Comprobación de que todo funciona correctamente</b>                      | Hito 3                           |
|       | S4     | <b>Entrega Hito 3:</b> Vídeo final (con voces), Memoria Completa (ICOM).              | <b>Hito 3 (14/05)</b>            |

## 8.- Pruebas y validación

Para cada servicio crítico gestionado por NetManager, se han ejecutado las siguientes pruebas a través de la interfaz web:

| Servicio / Tarea a Validar | Prueba Realizada                 | Resultado Esperado              | Resultado Real                 | Estado |
|----------------------------|----------------------------------|---------------------------------|--------------------------------|--------|
| Módulo Core (Flask)        | Acceso a la IP del servidor.     | Carga de la Dashboard.          | Dashboard carga rápido.        | OK     |
| DHCP                       | Guardar nueva subred.            | Archivo dhcpd.conf actualizado. | Archivo actualizado con datos. | OK     |
| DHCP (Validación Real)     | Arrancar cliente (VM) en la red. | Asignación de IP dinámica.      | Cliente recibe IP del rango.   | OK     |
| FTP (vsftpd)               | Crear usuario test_user.         | Usuario añadido en Ubuntu.      | Usuario creado sin shell.      | OK     |

## 9.- Relación del proyecto con los módulos del ciclo

### SISTEMAS OPERATIVOS EN RED

- **Competencia aplicada:** Administración avanzada de servidores Linux.
- **Justificación técnica:** NetManager se ha desarrollado sobre un servidor Ubuntu Server 22.04 LTS (Jammy). El proyecto ha requerido la configuración de permisos avanzados mediante el archivo /etc/sudoers para que el backend (Flask) ejecute comandos de sistema (useradd, chpasswd, systemctl), y la gestión de usuarios y grupos locales para el servicio FTP.

### SERVICIOS EN RED

- **Competencia aplicada:** Despliegue y configuración de servicios de red.
- **Justificación técnica:** El núcleo del proyecto es la automatización de protocolos críticos. Se han implementado scripts para la configuración dinámica de /etc/dhcpd.conf (ISC-DHCP-Server) y la administración de /etc/vsftpd.conf (vsftpd). Además, se ha diseñado el Appliance para que resuelva DNS de forma autónoma, garantizando la operatividad del panel incluso en escenarios de contingencia de red.

## APLICACIONES WEB

- **Competencia aplicada:** Desarrollo y despliegue de soluciones web Full Stack.
- **Justificación técnica:** Se ha desarrollado una interfaz completa (Frontend en HTML5/CSS3) que se comunica con un servidor de aplicaciones (Backend en Python con Flask). Se ha programado la lógica para capturar peticiones HTTP (peticiones POST de formularios) y traducirlas a acciones de sistema, implementando un panel de control totalmente funcional y dinámico.

## SEGURIDAD Y ALTA DISPONIBILIDAD

- **Competencia aplicada:** Implementación de medidas de seguridad y preconfiguración ante fallos.
- **Justificación técnica:** El proyecto aborda la seguridad en múltiples niveles: el panel web implementa control de acceso, el servicio FTP usa chroot para aislar a los usuarios en sus directorios, y la concepción de Appliance offline mitiga riesgos externos como los ataques DDoS a repositorios globales (un punto clave validado en las pruebas de contingencia).

## 10.- Conclusiones

Tras terminar el proyecto NetManager PYME, hemos alcanzado los objetivos planteados inicialmente. En el desarrollo hemos usado nuestros conocimientos en la administración de sistemas, sino también la viabilidad de crear herramientas que humanicen y simplifiquen la gestión de redes complejas.

Las principales conclusiones extraídas de este proyecto son:

1. **Automatización como valor añadido:** Hemos comprobado que evitar el uso del terminal usando una interfaz Flask reduce drásticamente el error humano. Tareas que antes requerían saber modificar archivos de configuración ahora se realizan de forma segura y visual.
2. **Resolución de contingencias:** Durante el proyecto hemos tenido varios problemas, como la caída de repositorios globales de Ubuntu o problemas en la configuración DHCP y hemos aprendido a buscar soluciones, diagnosticando y tomando una solución más simple.
3. **Seguridad y Control:** Hemos aprendido la importancia de gestionar correctamente los privilegios de sistema. El uso de sudoers para que el servidor web interactúe con el sistema operativo se ha realizado bajo criterios de mínimos privilegios, garantizando que el servidor sea seguro y estable.
4. **Trabajo en equipo:** El desarrollo de NetManager nos ha ayudado a entender cómo se comportan diferentes elementos entre sí y como hacen sinergia (Sistemas, Backend y Diseño UI), pareciéndose a un entorno de trabajo real donde la integración de cada módulo es obligatoria para el éxito del producto final.

En resumen, NetManager es una solución funcional y profesional para PYMES, cumpliendo con la idea de que los mejores programas son los que son intuitivos y fácil de usar para el usuario final.

## 11.- Proyectos futuros

---

NetManager ha sido diseñado con una arquitectura modular que permite la expansión de sus funcionalidades. Debido a la priorización de la estabilidad del servicio FTP y el cumplimiento de los plazos del Hito 3, se han identificado las siguientes líneas de desarrollo para futuras versiones:

- **Implementación de Servidor Web (Apache):** La evolución más natural del proyecto es integrar la gestión de Virtual Hosts. Esto permitiría a las empresas no solo almacenar archivos vía FTP, sino también hostear sus propias páginas web o intranets directamente desde el panel, automatizando la creación de archivos de configuración
- **Estadísticas de Uso y Monitorización:** Integrar gráficas de consumo de CPU, memoria y tráfico de red en tiempo real en el panel principal.
- **Copias de Seguridad Automatizadas:** Crear una sección donde el usuario pueda programar backups de las carpetas de los usuarios FTP hacia un almacenamiento externo o la nube.

## 12.- Bibliografía/Webgrafía

---

Documentación oficial del Ubuntu Server 22.04 LTS (Jammy Jellyfish):  
<https://ubuntu.com/server/docs>

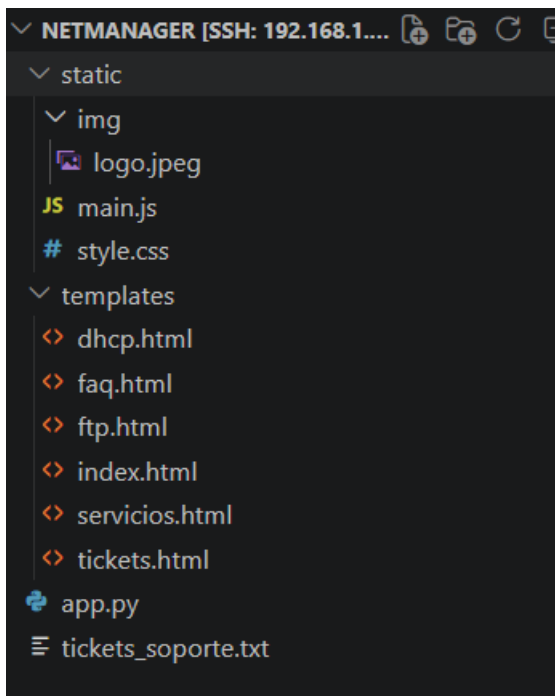
## 13.- Anexos

### Anexo I: Estructura del Proyecto (Directorios)

#### Anexo I: Estructura completa de directorios y archivos

La distribución jerárquica de ficheros que compone la infraestructura del sistema es la siguiente:  
netmanager

- app.py                      Cerebro del servidor (Código Backend en Python)
- tickets\_soporte.txt        Fichero de base de datos local en texto plano
- templates/                Pantallas de la interfaz (Estructura HTML)
  - index.html                Panel de control y Dashboard principal
  - dhcp.html                Configuración de direccionamiento de red
  - ftp.html                  Gestión y altas/bajas del servicio FTP
  - servicios.html            Tabla de monitorización de sockets y puertos
  - faq.html                  Manual técnico de preguntas frecuentes
  - tickets.html              Formulario de creación de incidencias
- static/                    Archivos visuales y lógicos del navegador
  - style.css                Diseño, colores y estilos corporativos (CSS)
  - main.js                  Lógica del cliente y ventanas modales (JavaScript)
- img/
  - logo.jpeg                Imagen de marca del proyecto





## Anexo II: Fragmento de Código principal (Logic Backend)

Este es el bloque de código principal del archivo app.py. Este fragmento es el encargado de recibir los datos del formulario web y sobrescribir el archivo de configuración del servidor DHCP de ISC (/etc/dhcp/dhcpd.conf).

```
@app.route('/configurar/dhcp', methods=['GET', 'POST'])
def configurar_dhcp():
    if request.method == 'GET':
        #Formulario
        return render_template('dhcp.html')

    elif request.method == 'POST':
        # Recogida de datos
        subred = request.form['subred']
        mascara = request.form['mascara']
        rango_inicio = request.form['rango_inicio']
        rango_fin = request.form['rango_fin']
        router = request.form['router']

        # Creamos configuración para archivo de servidor
        nueva_configuracion = f"""
# Generado por NetManager
default-lease-time 600;
max-lease-time 7200;
authoritative;

subnet {subred} netmask {mascara} {{
    range {rango_inicio} {rango_fin};
    option routers {router};
    option domain-name-servers 8.8.8.8, 8.8.4.4;
}}
"""
```

